



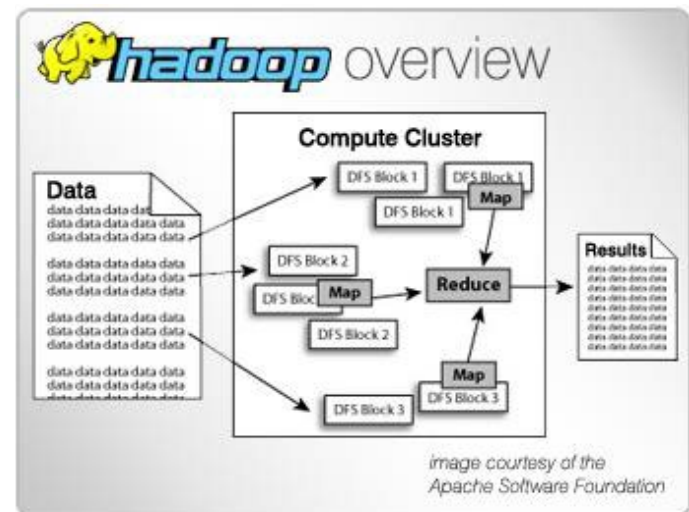
Introduction to Hadoop (MR₁)

Dongchul Park

Chung-Ang University

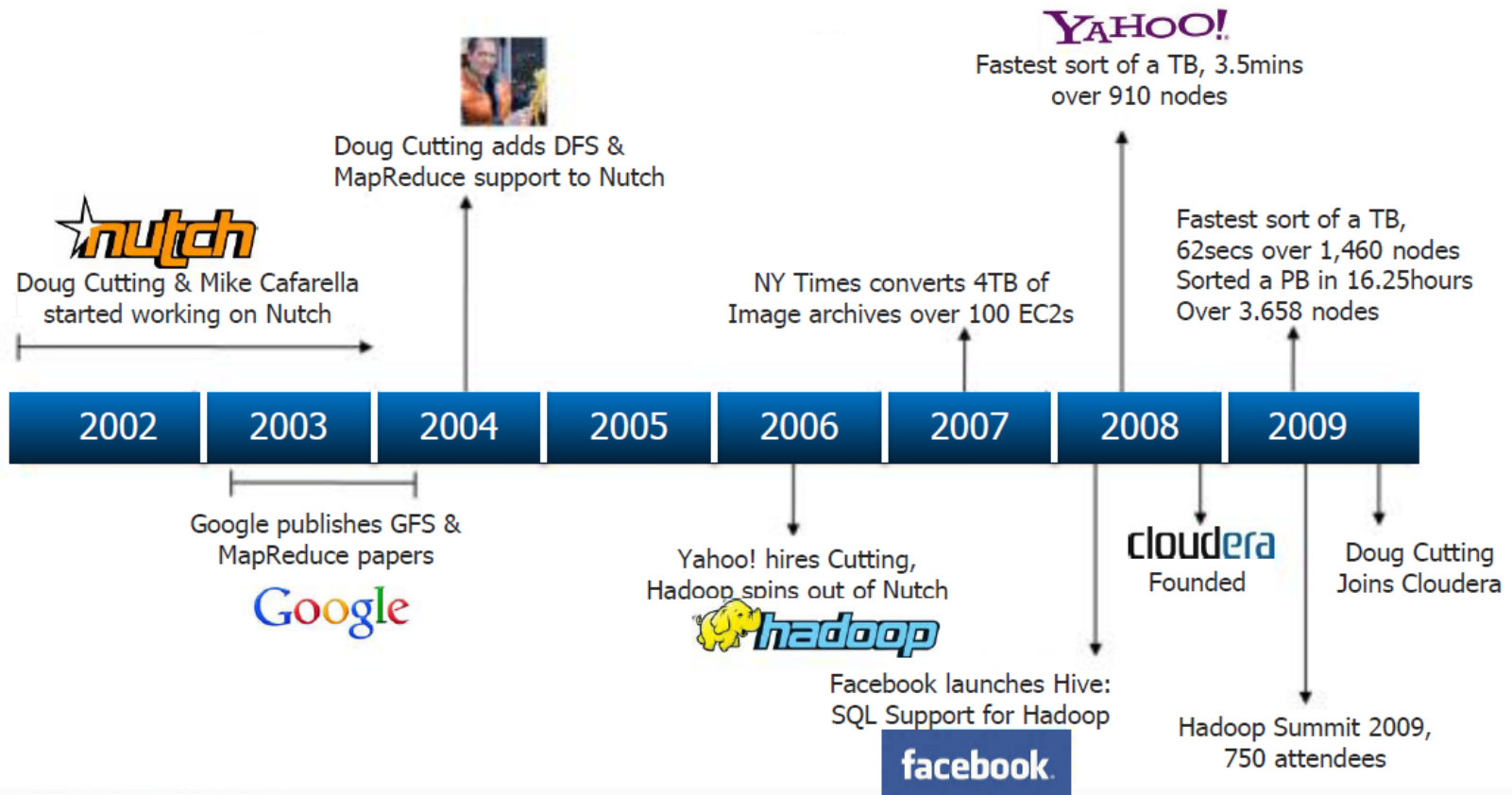
What is Hadoop?

- Open-source data storage and processing API
- Massively scalable, automatically parallelizable
 - Based on work from Google
 - GFS + MapReduce + BigTable
 - Current Distributions based on Open Source and Vendor Work
 - Apache Hadoop
 - Cloudera – CDH4
 - Hortonworks
 - MapR (now, HPE)
 - AWS
 - Windows Azure HDInsight



Hadoop History

Hadoop History



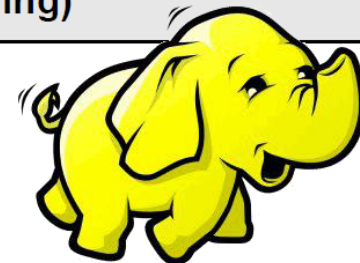
Why Use Hadoop?



- Cheaper
 - Scales to Petabytes or more
- Faster
 - Parallel data processing
- Better
 - Suited for particular types of BigData problems

RDBMS vs. Hadoop

	Traditional RDBMS	Hadoop / MapReduce
Data Size	Gigabytes (Terabytes)	Petabytes (Hexabytes)
Access	Interactive and Batch	Batch – NOT Interactive
Updates	Read / Write many times	Write once, Read many times
Structure	Static Schema	Dynamic Schema
Integrity	High (ACID)	Low
Scaling	Nonlinear	Linear
Query Response Time	Can be near immediate	Has latency (due to batch processing)

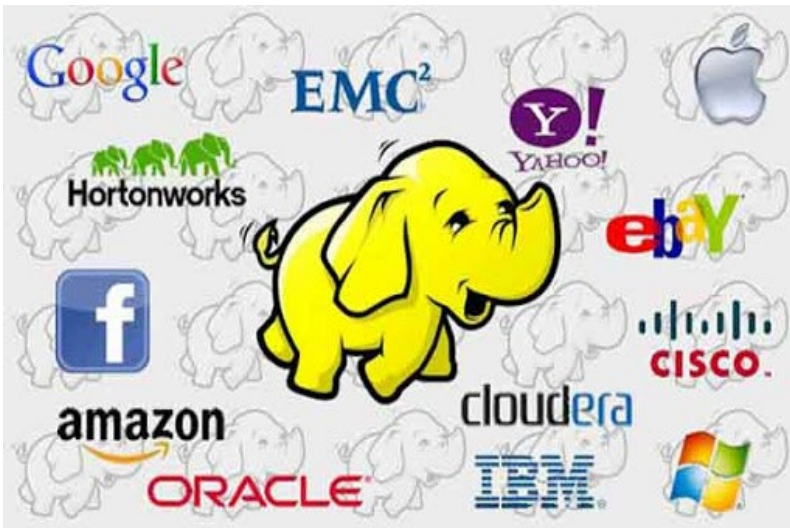


ACID: Atomicity, Consistency, Isolation, Durability

Where is Hadoop used?

Industry	Use Cases
Technology	Search People you may know Movie recommendations
Banks	Fraud Detection Regulatory Risk management
Media Retail	Marketing analytics Customer service Product recommendations
Manufacturing	Preventive maintenance

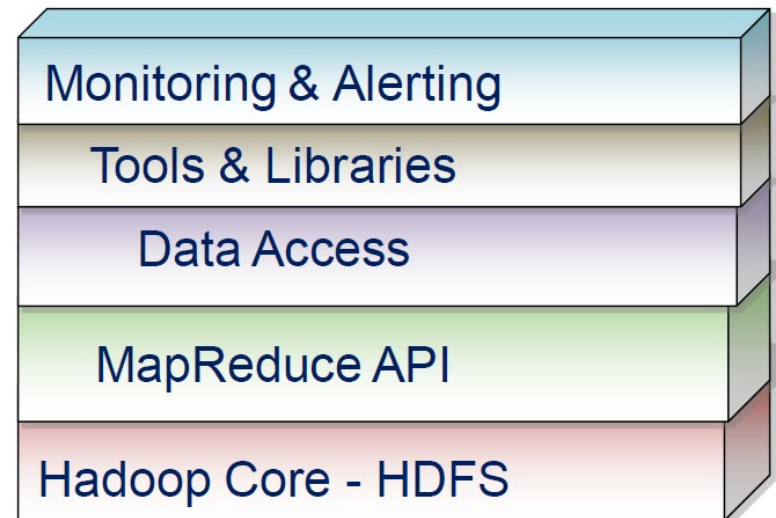
Companies Using Hadoop



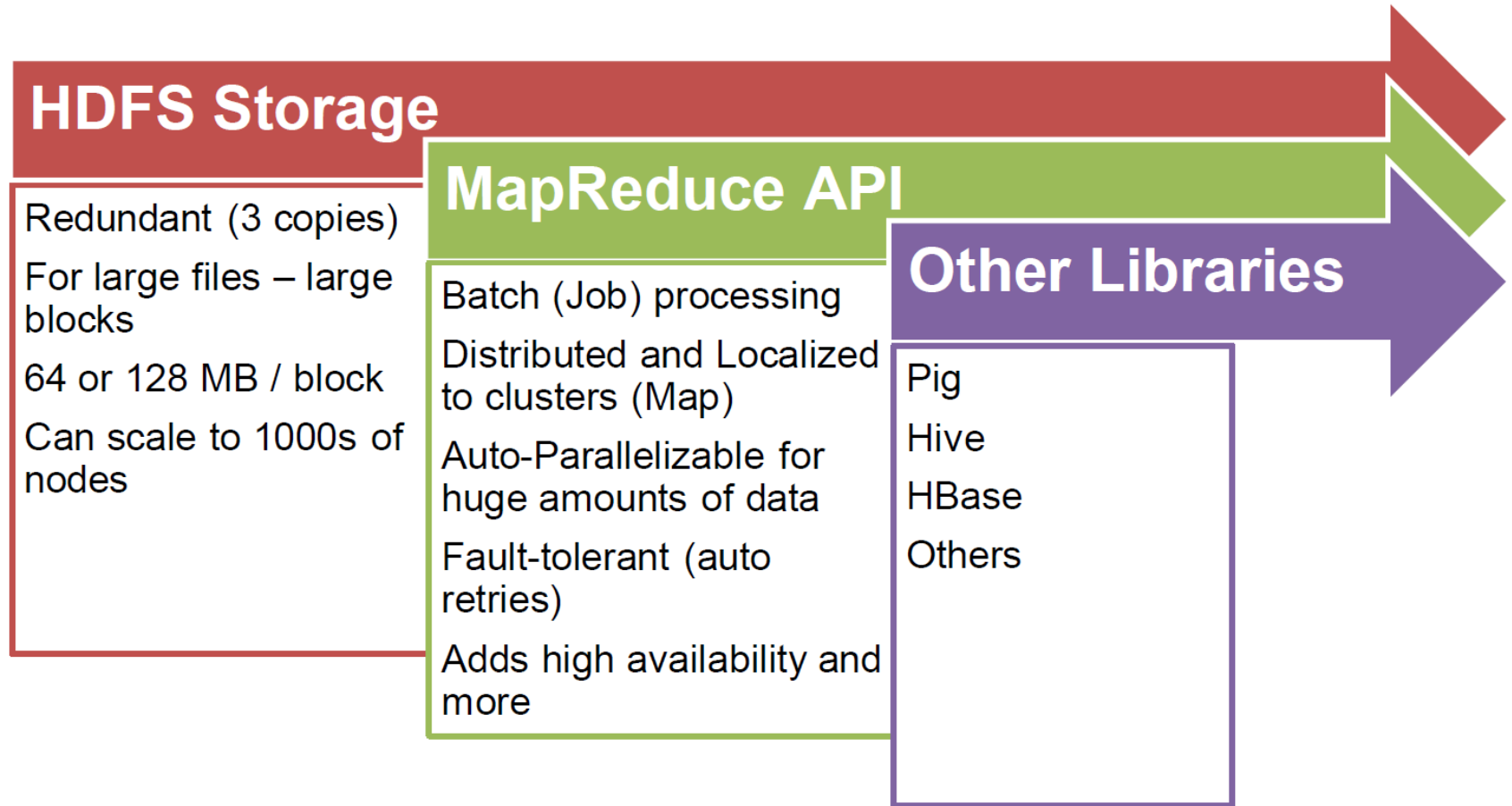
- Search
 - Yahoo, Amazon, Zvents
- Log Processing
 - Facebook, Yahoo, ContextWeb.Joost, Last.fm
- Recommendation Systems
 - Facebook, LinkedIn
- Data Warehouse
 - Facebook, AOL
- Video & Image Analysis
 - New York Times, Eyealike
- ----- Almost in every domain

Hadoop is a set of Apache Frameworks and more...

- Data storage (HDFS)
 - Runs on commodity hardware (usually Linux)
 - Horizontally scalable (scale-out)
- Processing (MapReduce)
 - Parallelized (scalable) processing
 - Fault tolerant
- Other Tools / Frameworks
 - Data Access
 - HBase, Hive, Pig, Mahout
 - Tools
 - Hue, Sqoop, Flume
 - Monitoring
 - Greenplum, Cloudera



Core parts of Hadoop distribution



Hadoop Cluster HDFS Storage

One Name Node

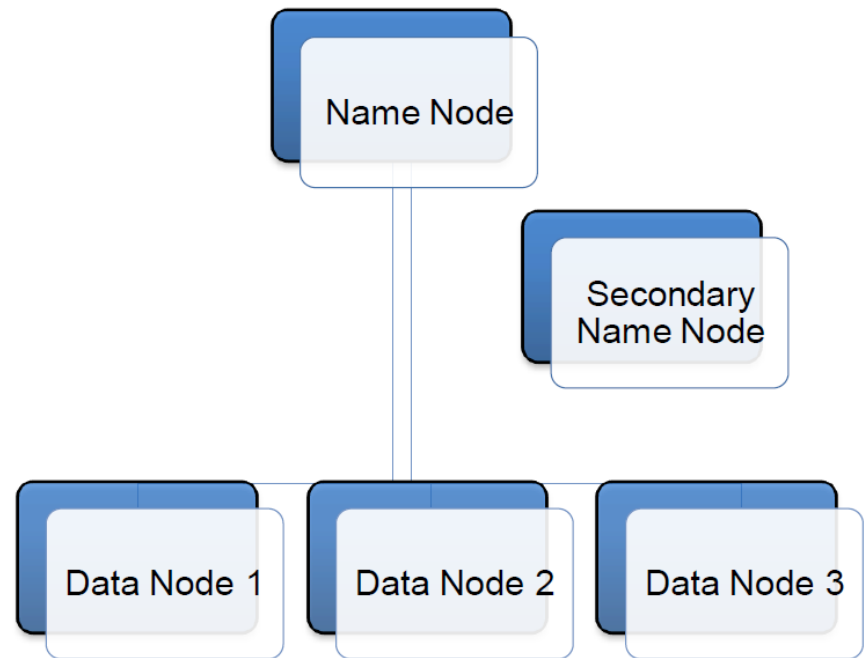
- Contains web site to view cluster information
- V2 Hadoop uses multiple Name Nodes for HA

Many Data Nodes

- 3 copies of each node by default

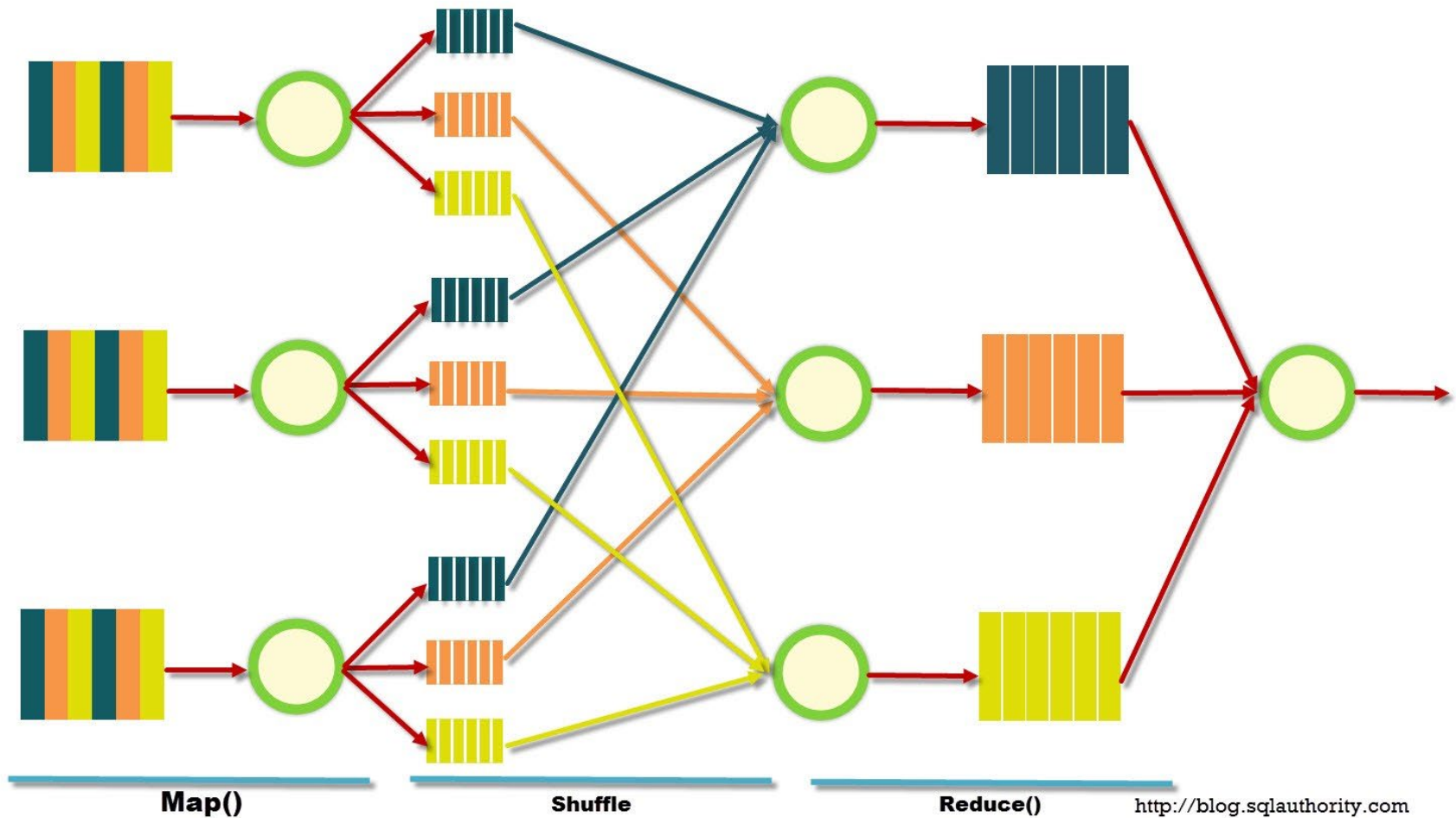
Work with data in HDFS

- Using common Linux shell commands
- Block size is 64 or 128 MB

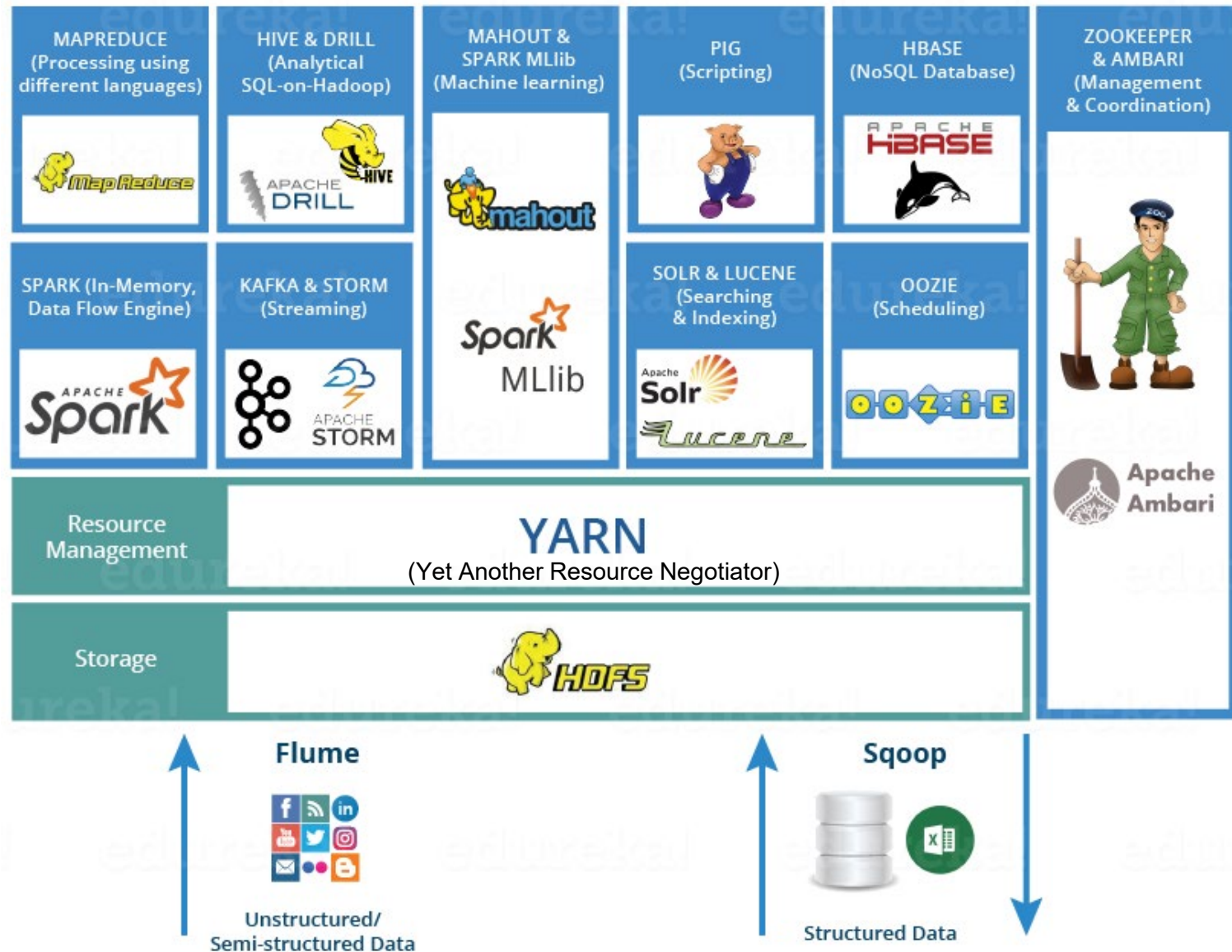


MapReduce Job: Logical View

How MapReduce Works?



Hadoop Ecosystem

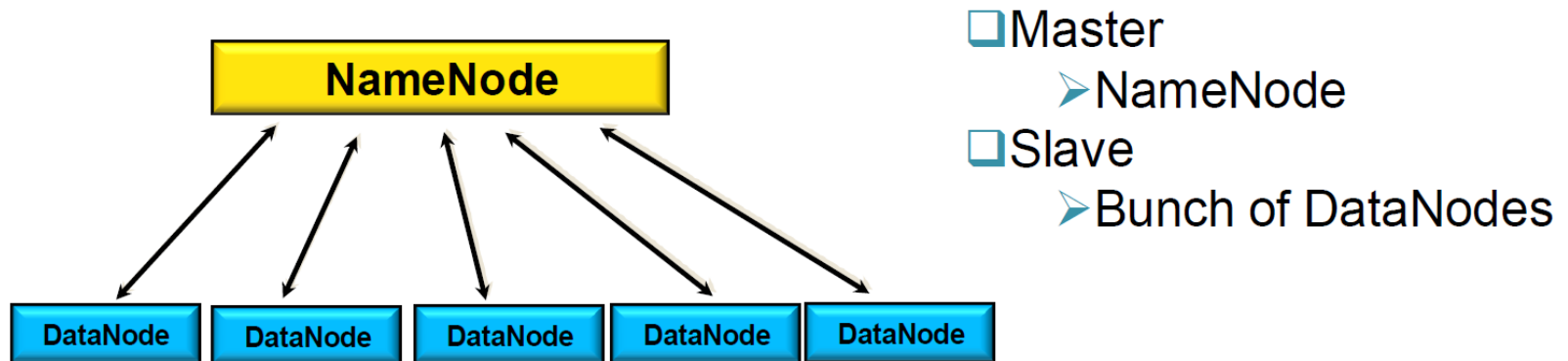


Common Hadoop Distributions

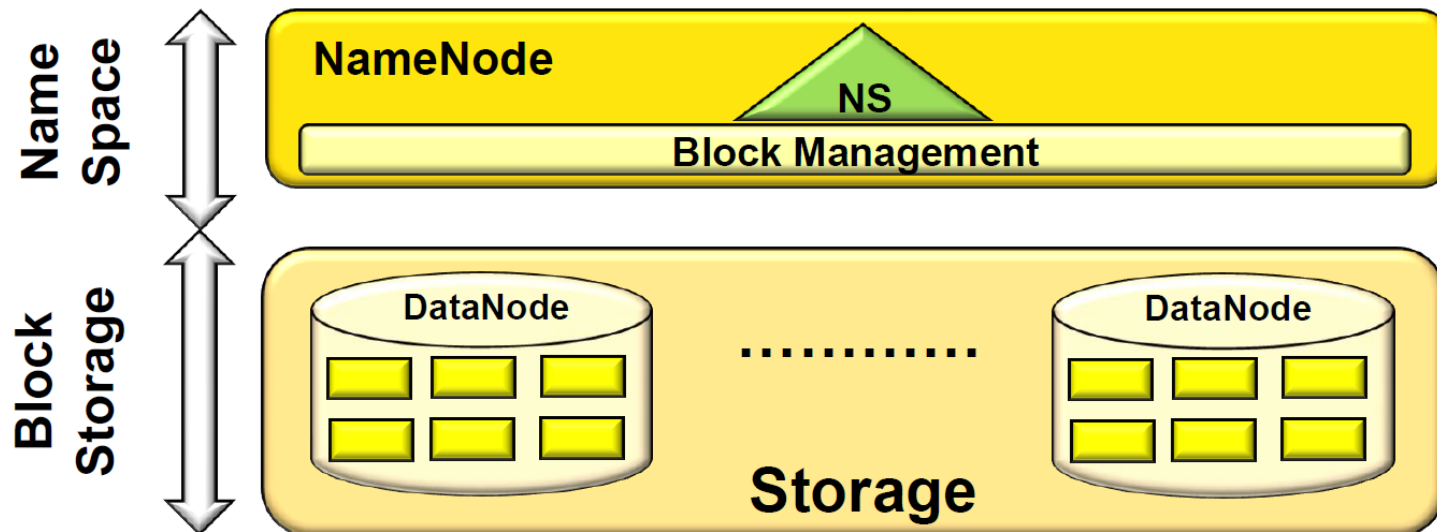
- Open Source
 - Apache
- Commercial
 - Cloudera
 - Hortonworks
 - MapR
 - AWS MapReduce
 - Microsoft HDInsight



HDFS: Architecture



HDFS Layers



HDFS: Basic Features

Highly fault-tolerant

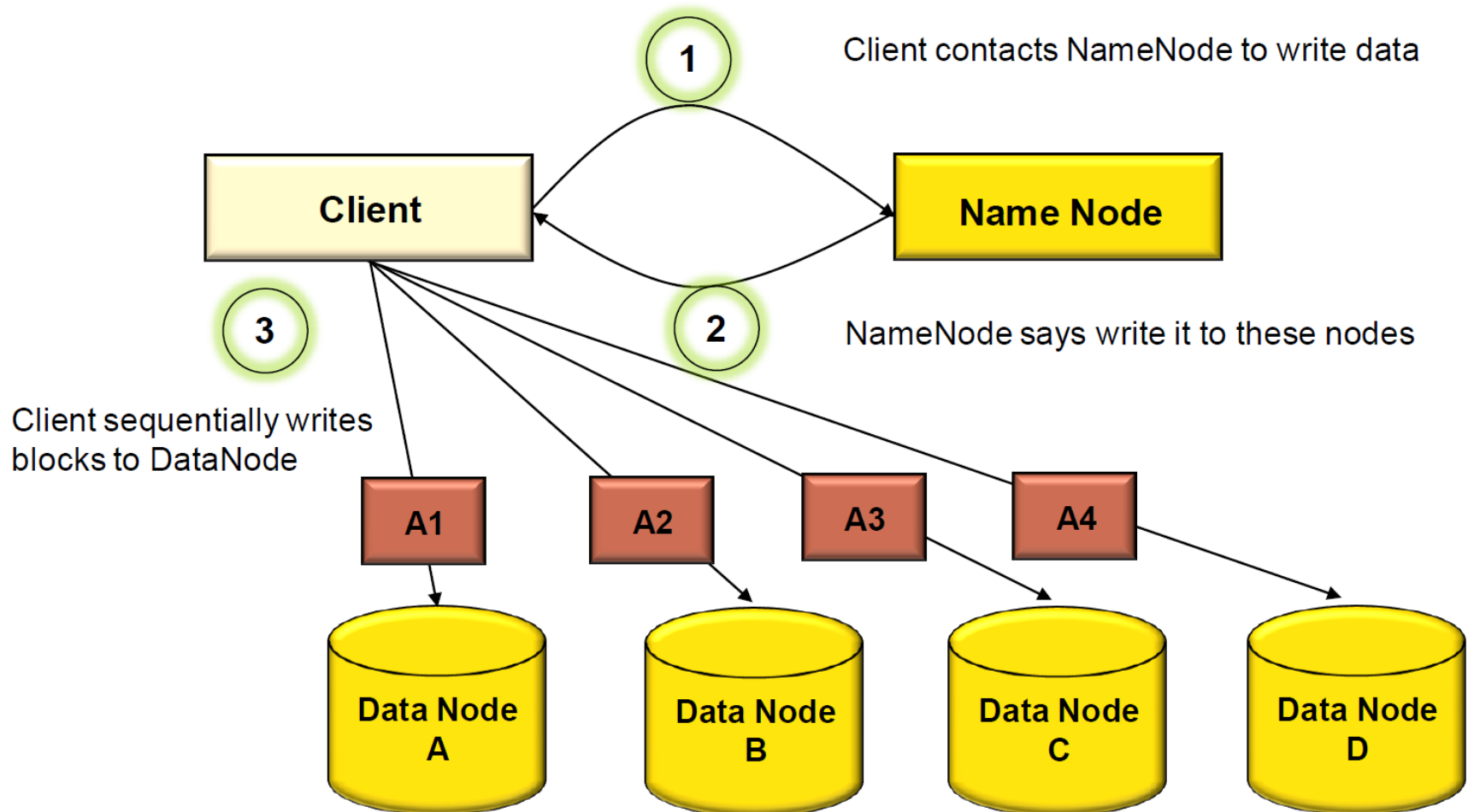
High throughput

Suitable for applications with large data sets

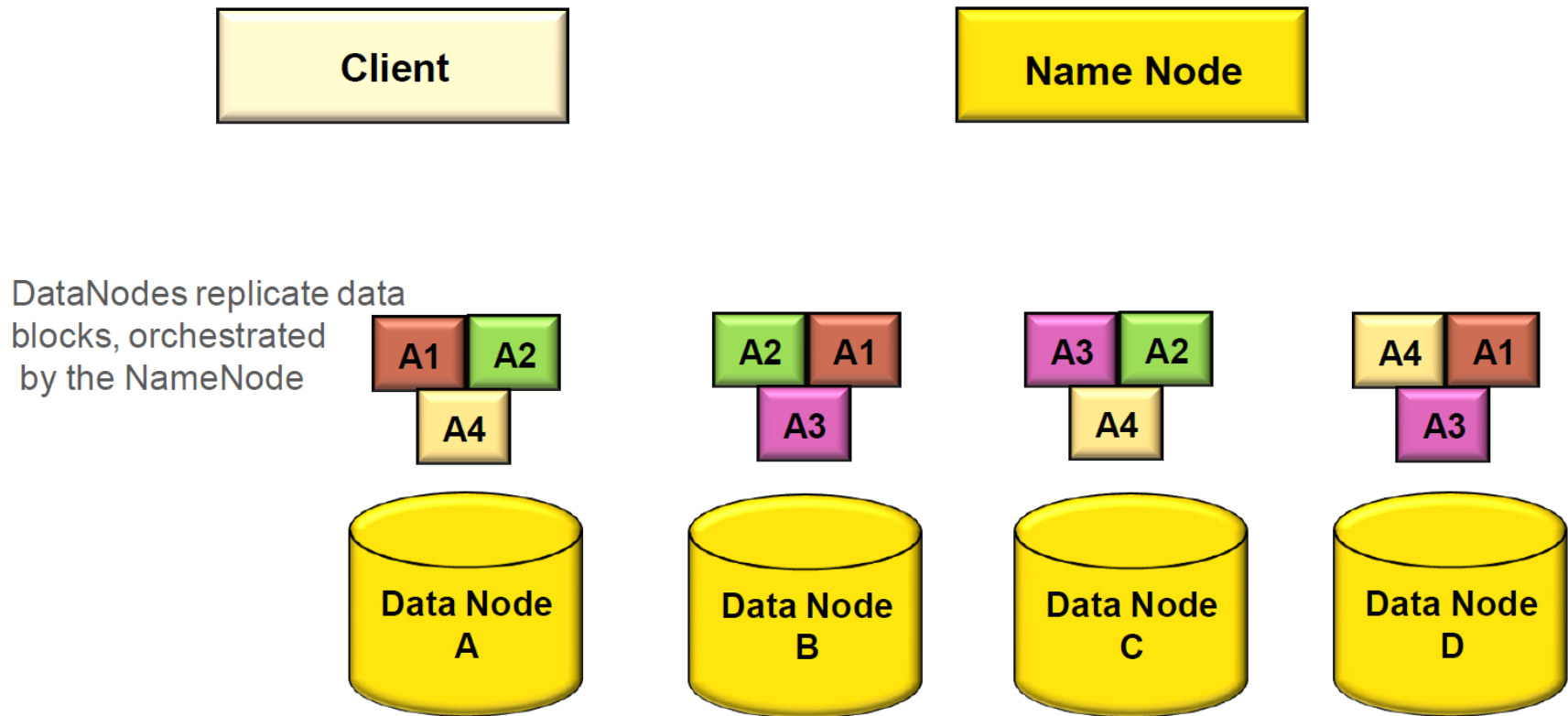
Streaming access to file system data

Can be built out of commodity hardware

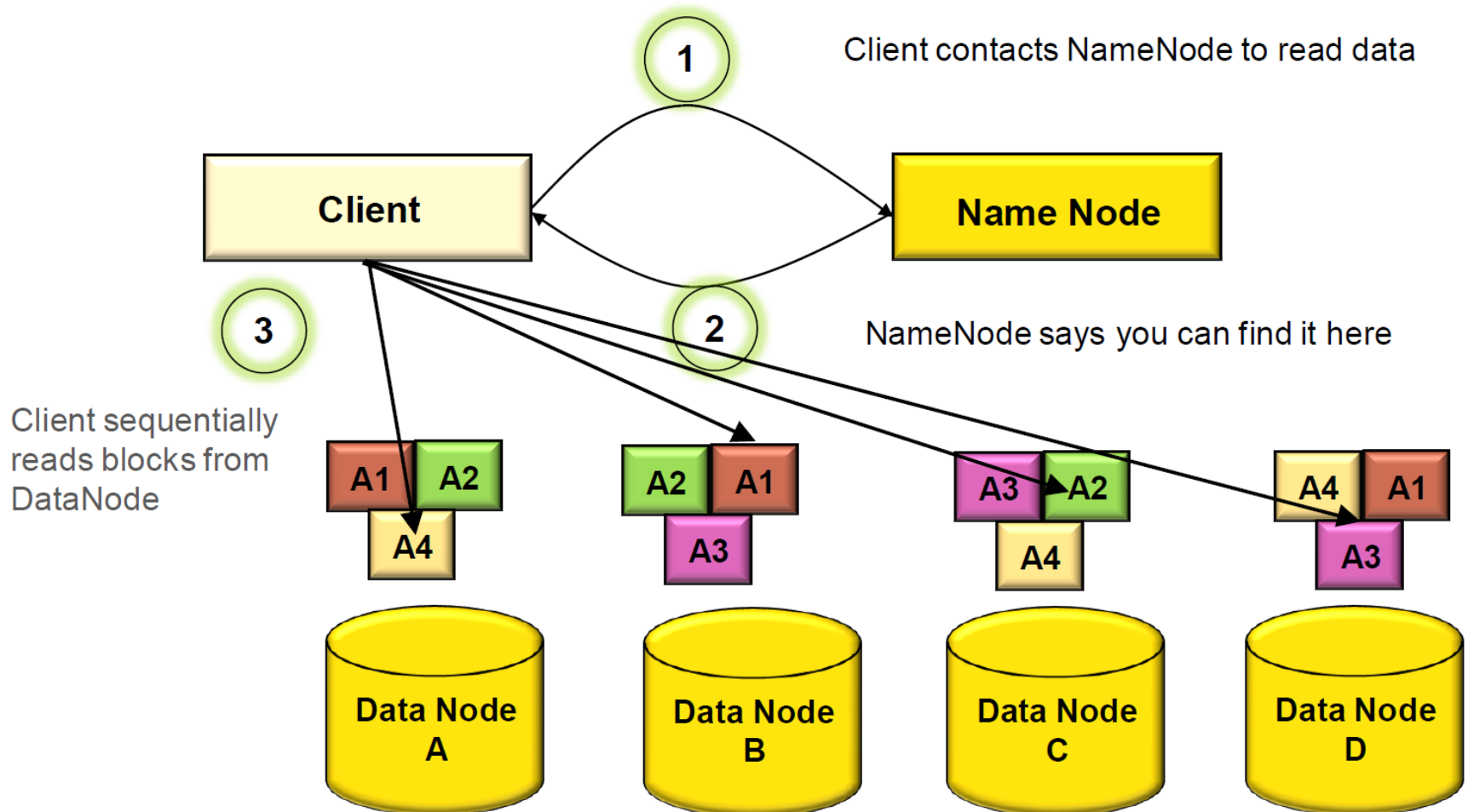
HDFS Write (1/2)



HDFS Write (2/2)

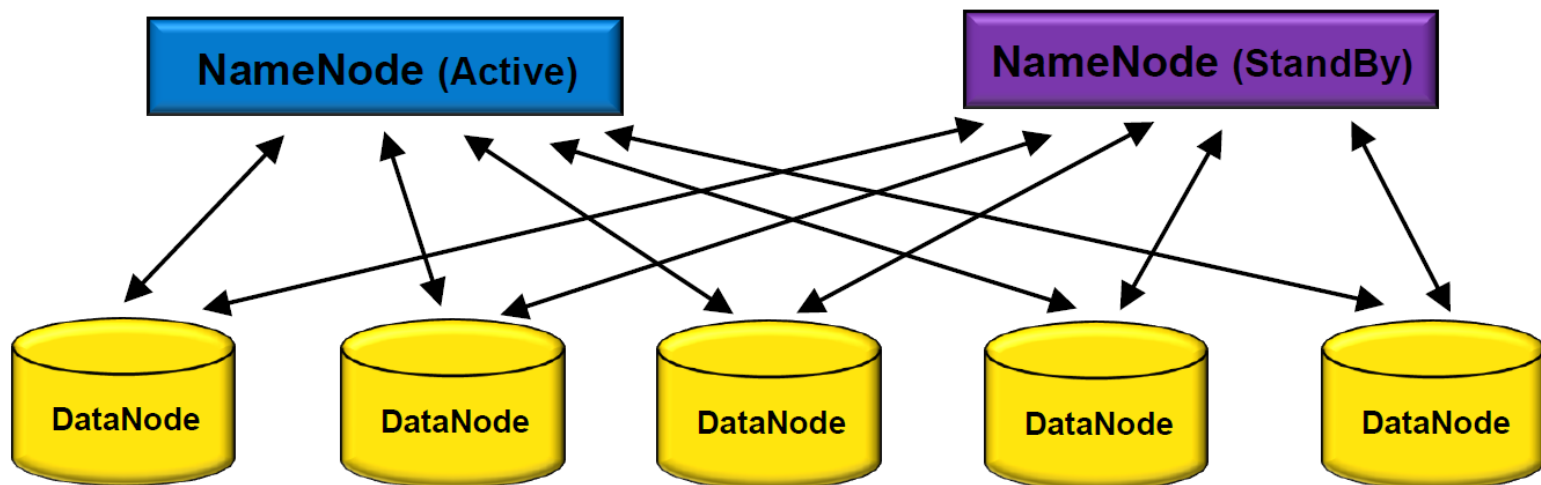


HDFS Read



HA (High Availability) for NameNode

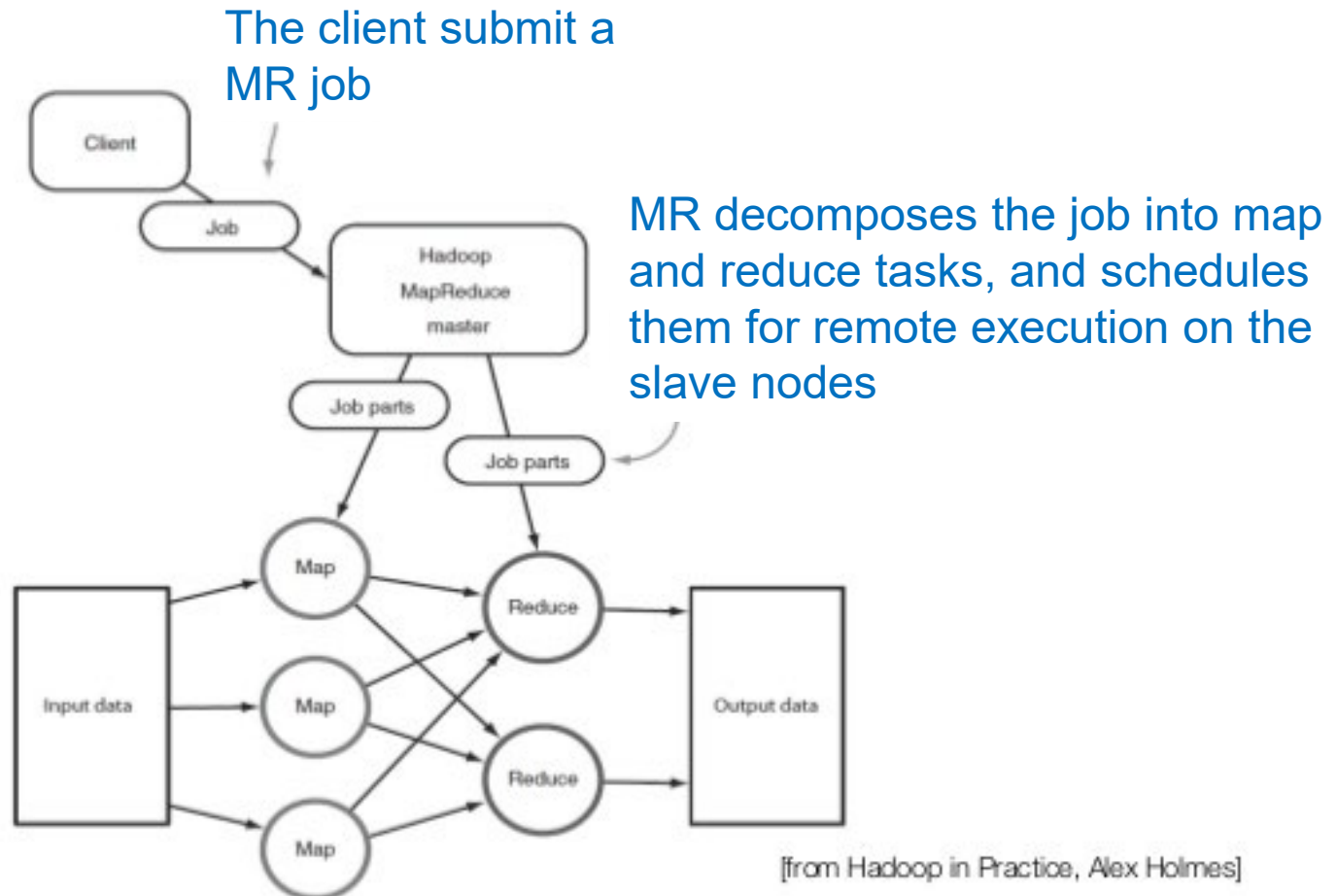
- Active NameNode
 - Do normal namenode's operation
- Standby NameNode
 - Maintain NameNode's data
 - Ready to be active NameNode



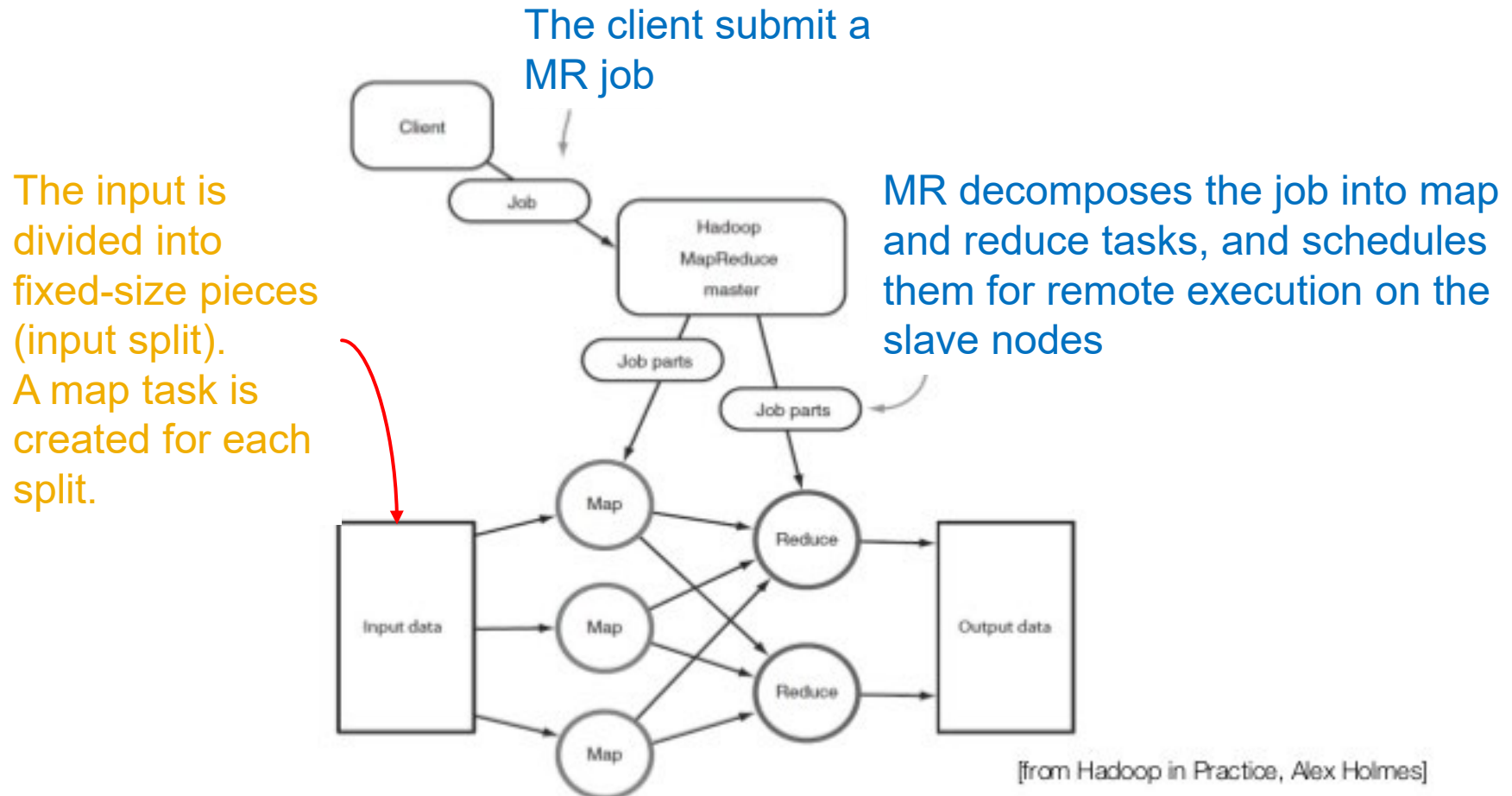
MapReduce

- A batch based, distributed computing framework modelled on Google's MapReduce paper.
- MapReduce decomposes work into small parallelized map and reduce tasks which are scheduled for remote execution on slave nodes.
- Terminology
 - Job: a full program
 - Task: the execution of a single map/reduce work over data split.
 - Mapper: a map task
 - Reducer: a reduce task
- Works only on KEY/VALUE pair format
 - Map(key1, value1) → list(key2, value2)
 - Reduce(key2, list(value2)) → (key2, value3)

A MapReduce Job

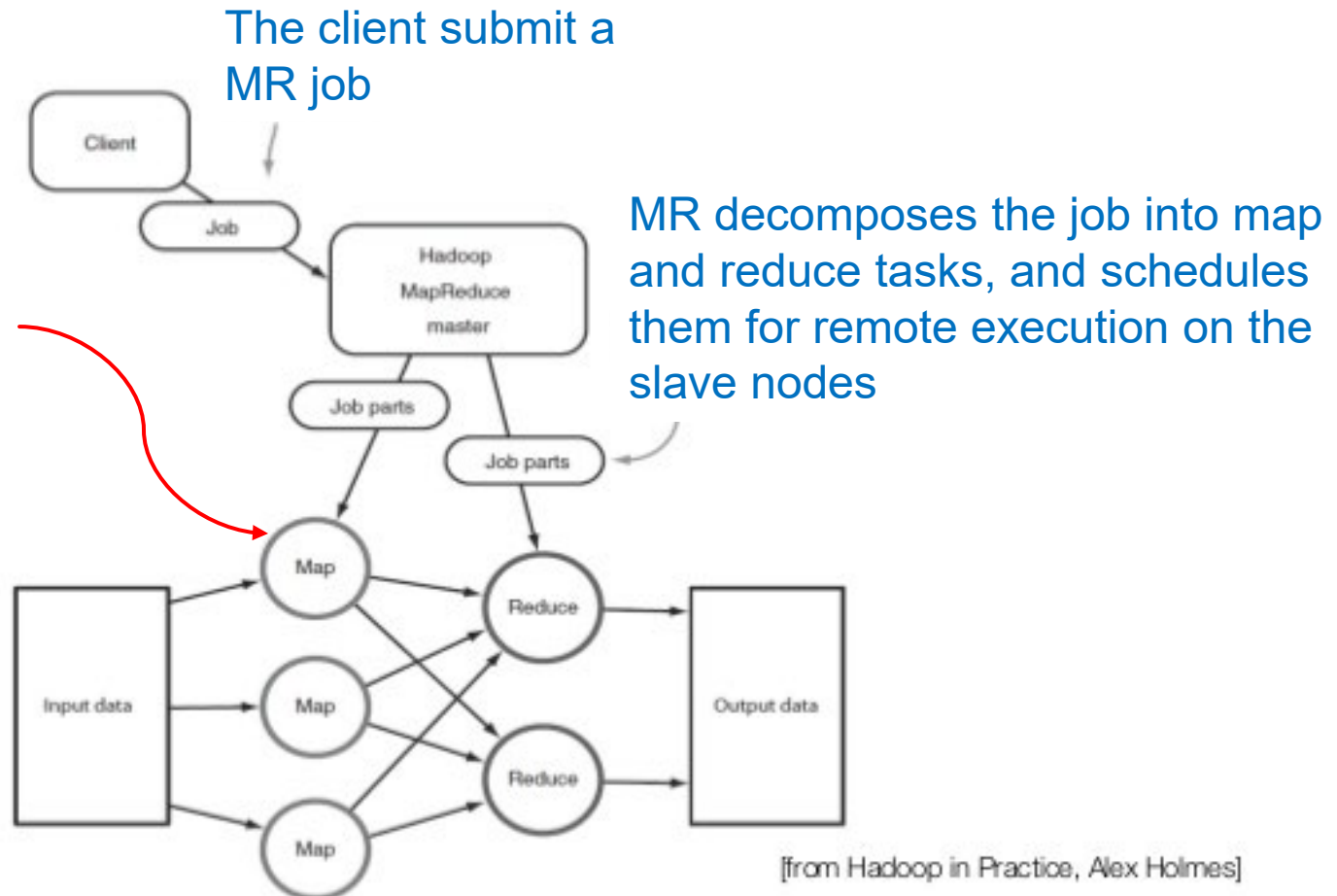


A MapReduce Job



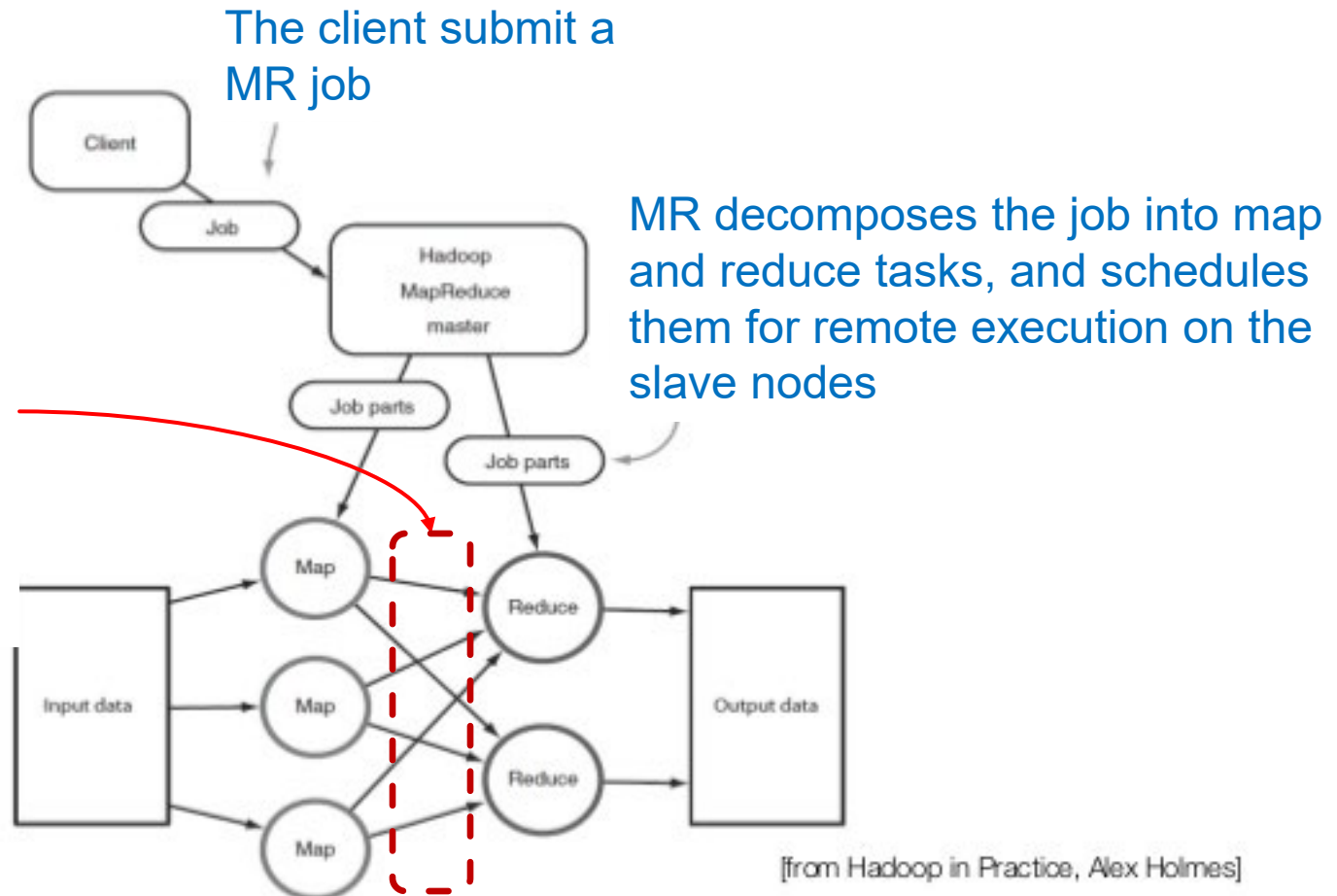
A MapReduce Job

The role of the programmer is to define the map and reduce functions

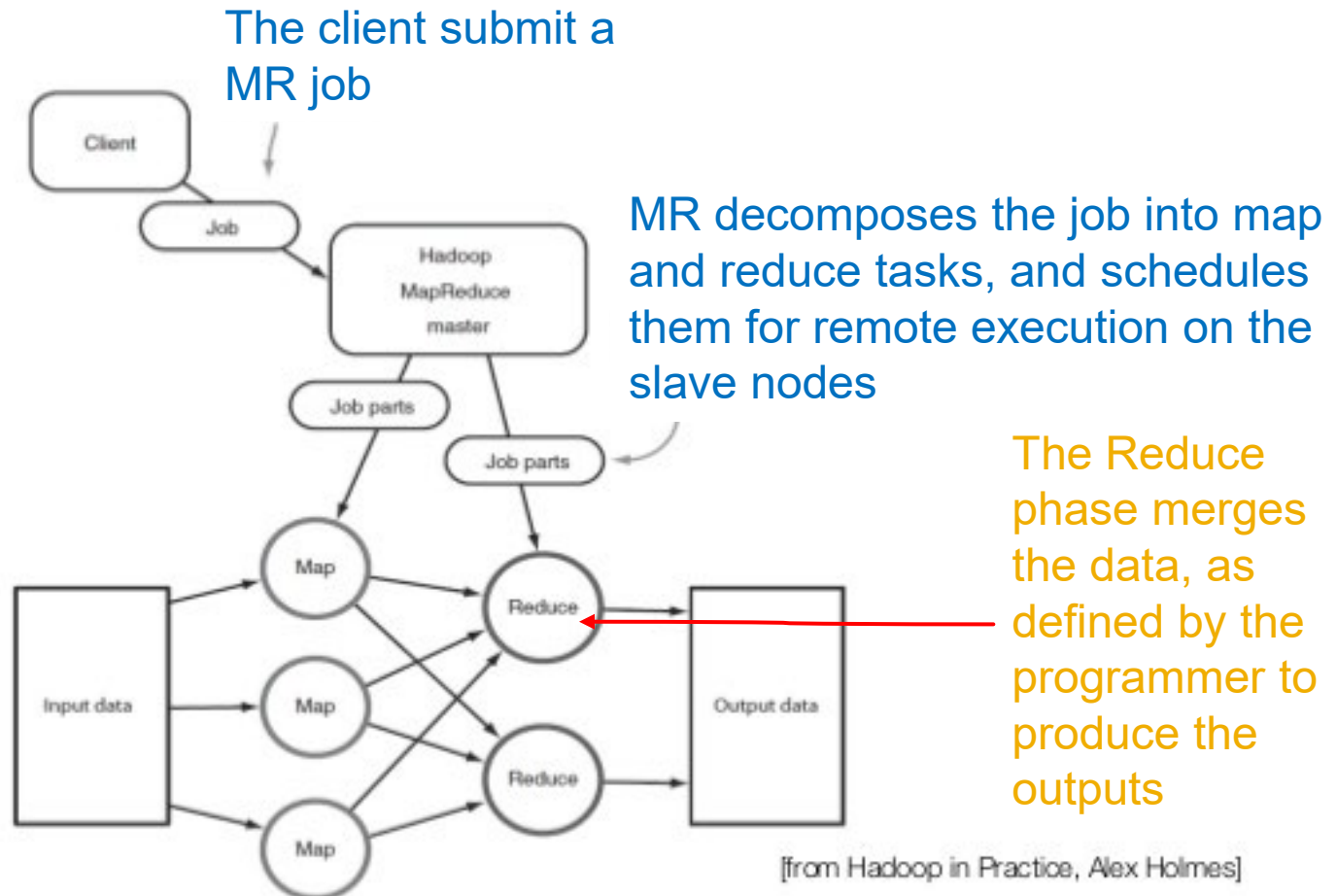


A MapReduce Job

The shuffle & sort phases between Map and Reduce combine map outputs and sort them for the reducers



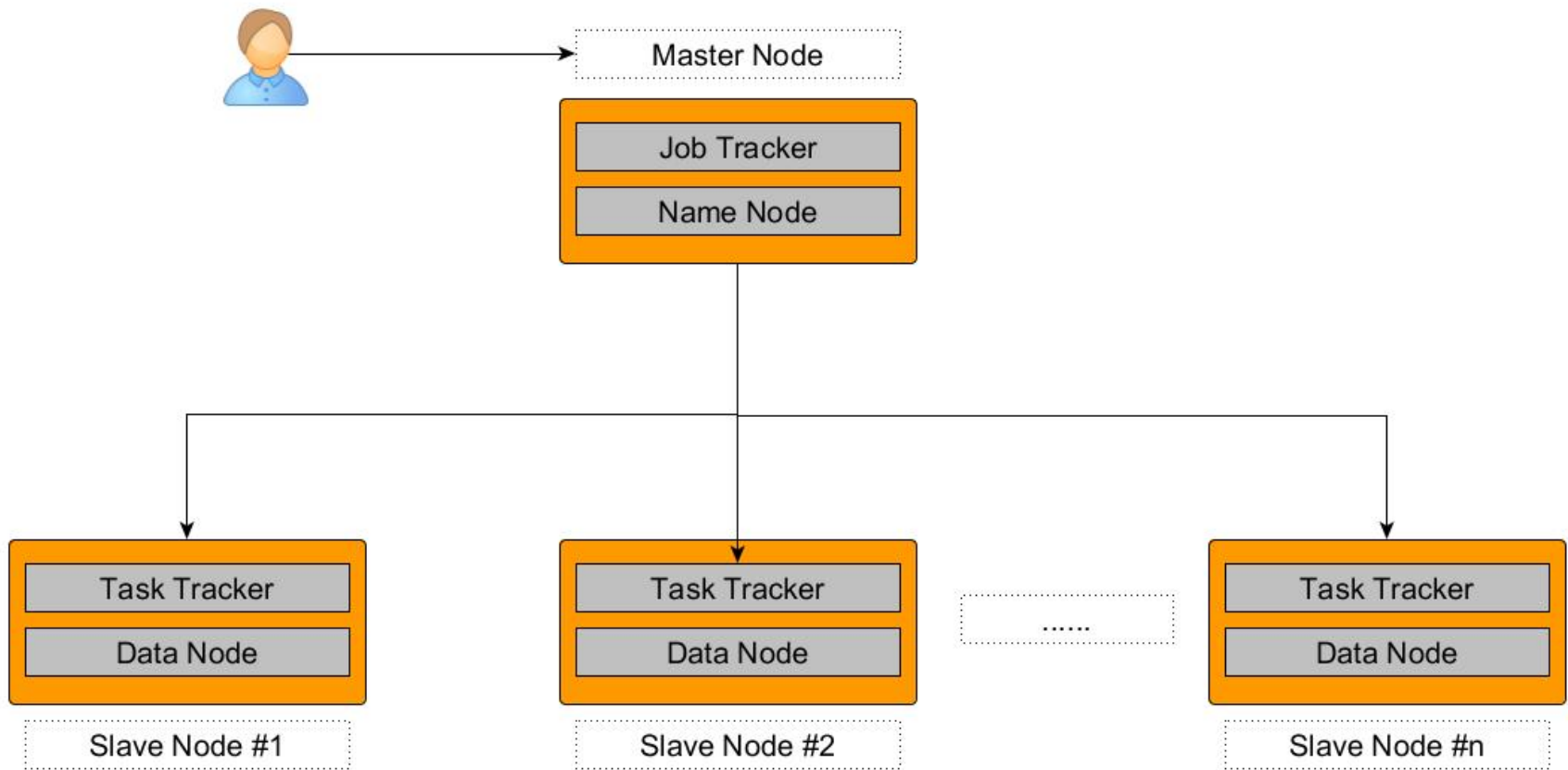
A MapReduce Job



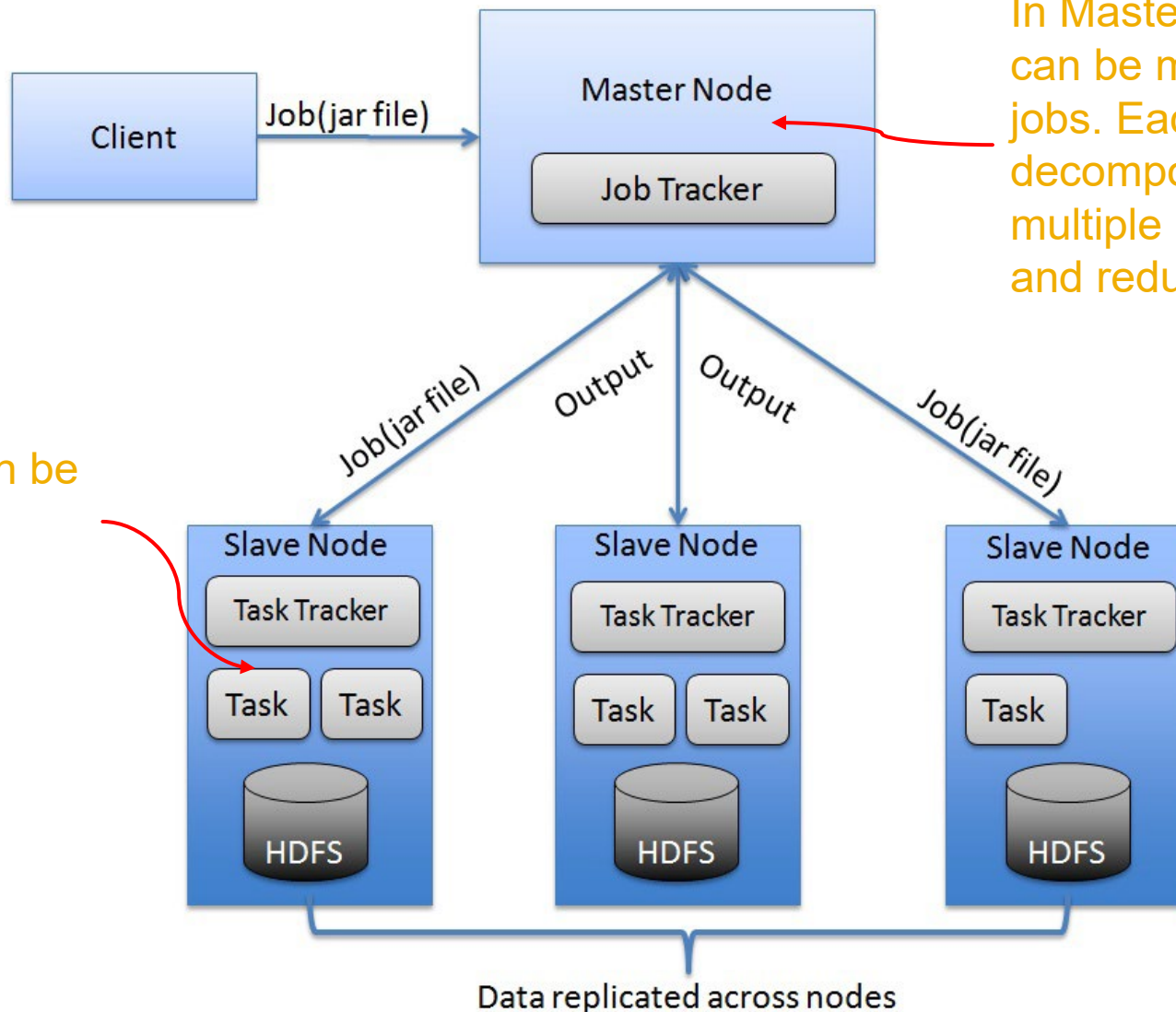
Hadoop MR components

- Master: JobTracker
 - Controls MapReduce jobs
 - Assigns Map & Reduce tasks to TaskTracker on the other nodes of the cluster
 - Monitors the tasks as they are running
 - Relaunches failed tasks on other nodes in the cluster
- Slave: TaskTracker
 - A single TaskTracker per slave node
 - Manages the execution of the individual tasks on the node
 - Can instantiate many JVMs to handle tasks in parallel
 - Communicates back to the JobTracker via a heartbeat

Hadoop Architecture



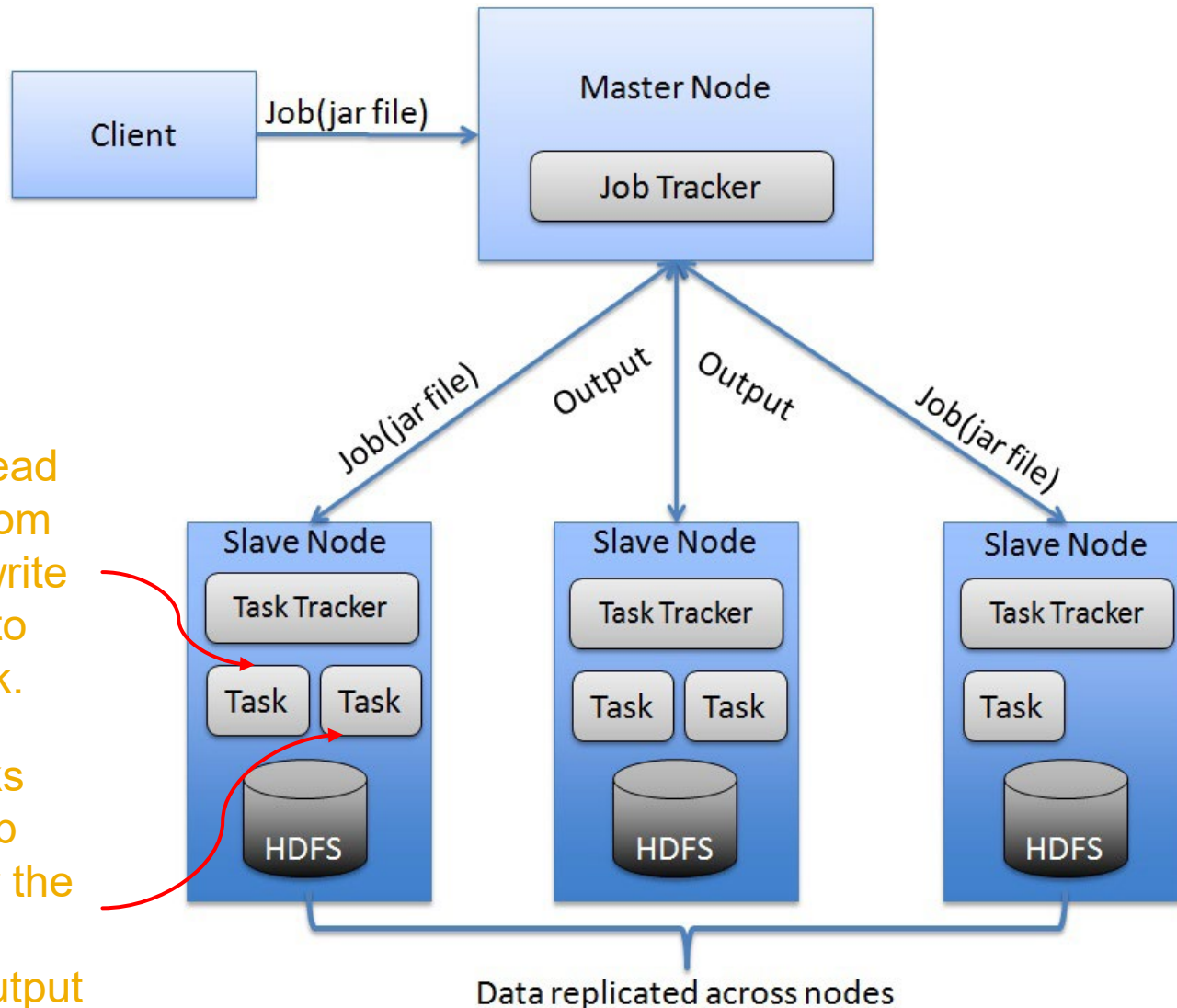
Hadoop MR Architecture



In Master node, there can be multiple active jobs. Each job decomposes into multiple map tasks and reduce tasks.

This task can be map task or reduce task.

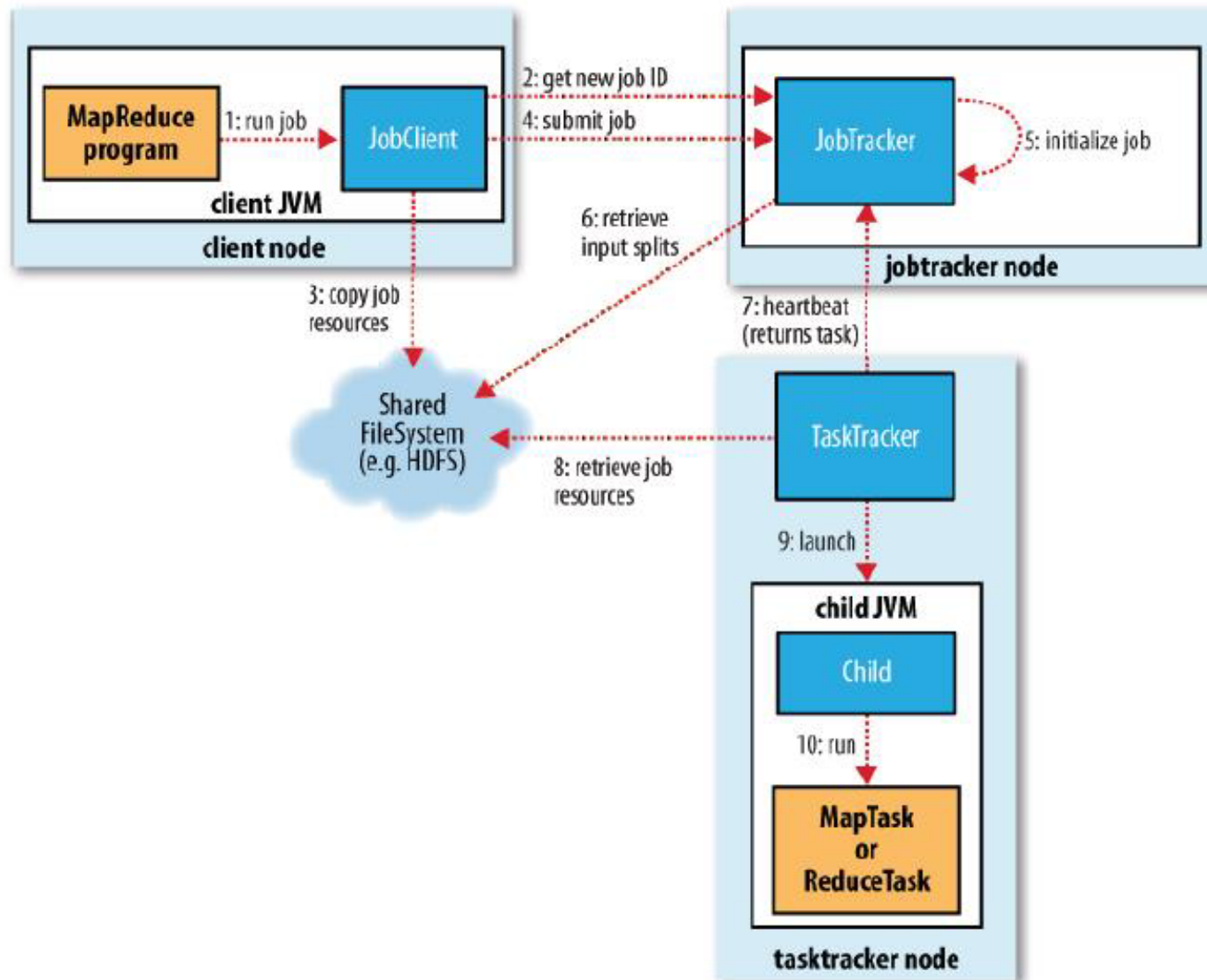
Hadoop MR Architecture



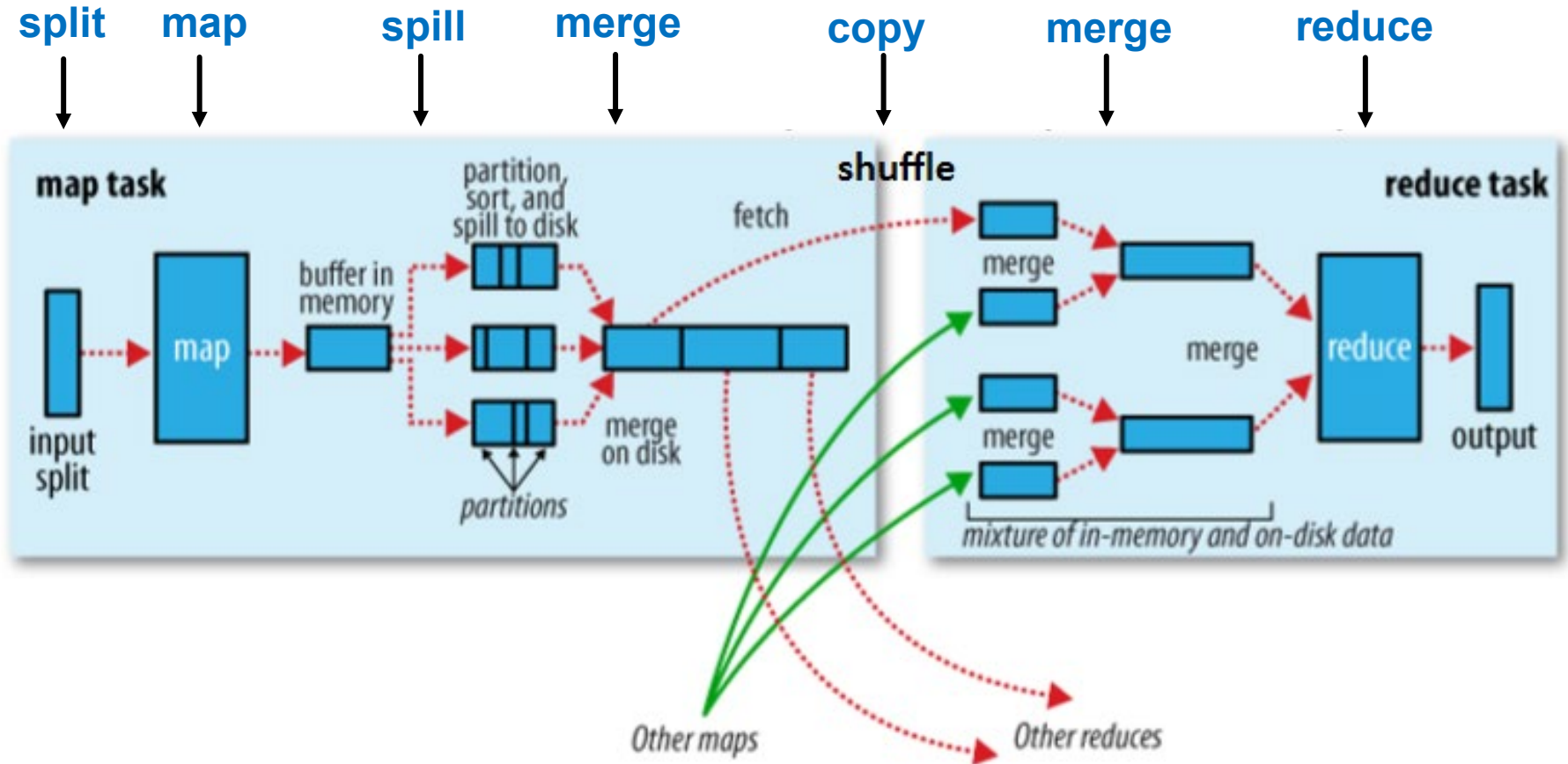
Map tasks read their input from HDFS and write their output to the local disk.

Reduce tasks read the map outputs over the network and write their output back to HDFS.

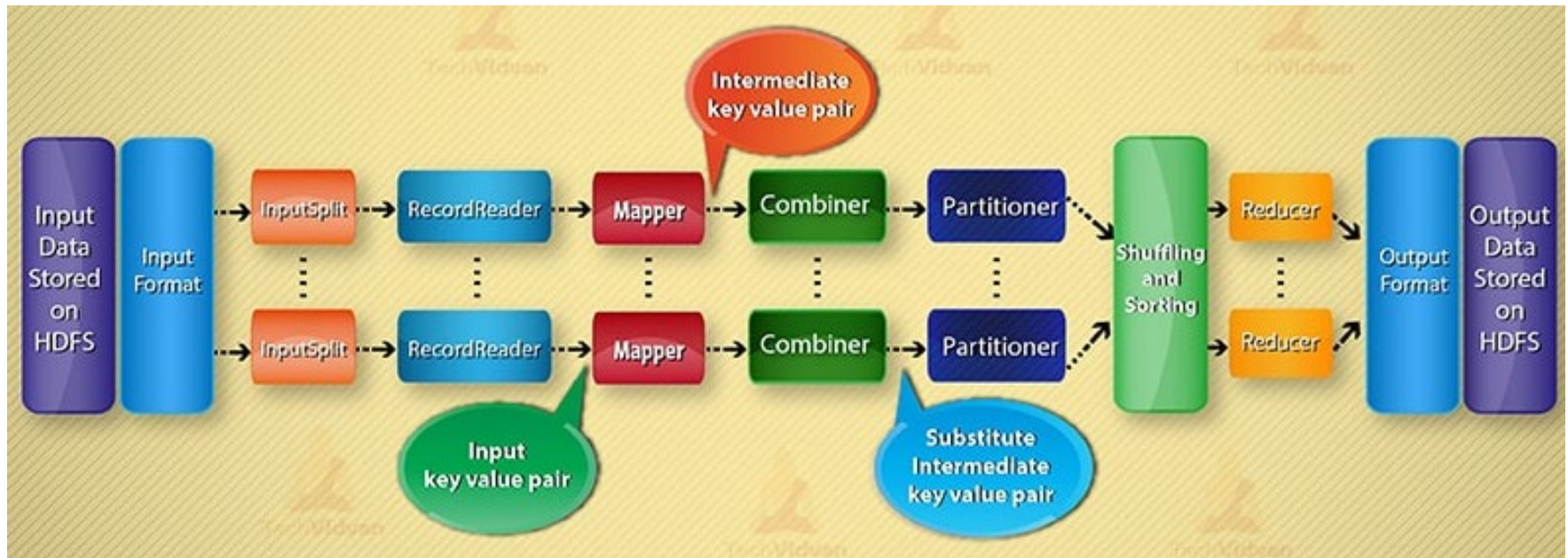
Job Execution Workflow (Hadoop v1)



Complete MapReduce execution



MapReduce Job Execution Flow



Bottlenecks?

- Reducers cannot start until all Mappers are finished and the output has been transferred to the Reducers and sorted.
- To alleviate the bottlenecks in Shuffle&Sort,
 - Hadoop starts to transfer data to the Reducers as the Mappers finish.
 - The percentage of Mappers which should finish before the Reducers start retrieving data is configurable.
- To alleviate bottlenecks caused by slow Mappers,
 - Hadoop uses **speculative execution**.
 - If a Mapper appears to be running significantly slower than the others, a new instance of the Mapper is started on another node, operating on the same data.
 - The result of the first finished Mapper will be used and the other one is terminated by Hadoop.



MapReduce V1 vs. V2

MapReduce v1 (MRv1): classic MR

- Architecture
 - JobTracker: single master daemon
 - Handles resource management, job scheduling, task monitoring and re-execution (if needed)
 - TaskTracker: slave daemon per worker node
 - Executes map and reduce tasks, reports status to JobTracker
- Workflow
 - Client submits job to JobTracker
 - JobTracker
 - Splits job into tasks and assigns tasks to TaskTrackers
 - TaskTrackers run map/reduce tasks
 - Report progress/status back to JobTracker
 - JobTracker tracks failures, reassigns tasks if needed
 - Once all task complete, job done

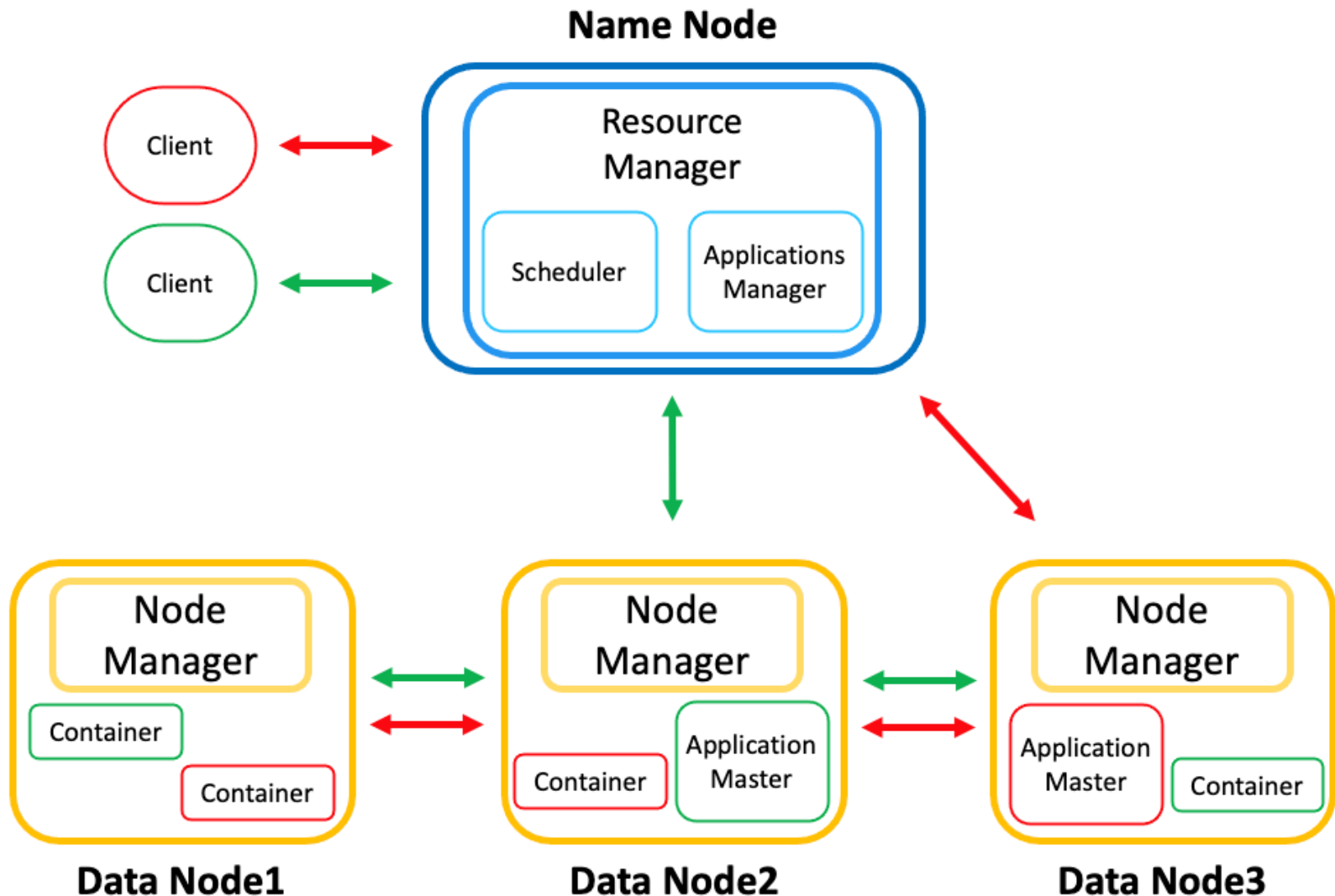
MapReduce v1 (MRv1)

- Limitations
 - Single JobTracker
 - Reliability and availability problems: single point of failure
 - Scalability bottleneck
 - One JobTracker manages every node (resources and scheduling)
 - About 4,000 nodes
 - Resources not flexible
 - Fixed map/reduce slots assignment

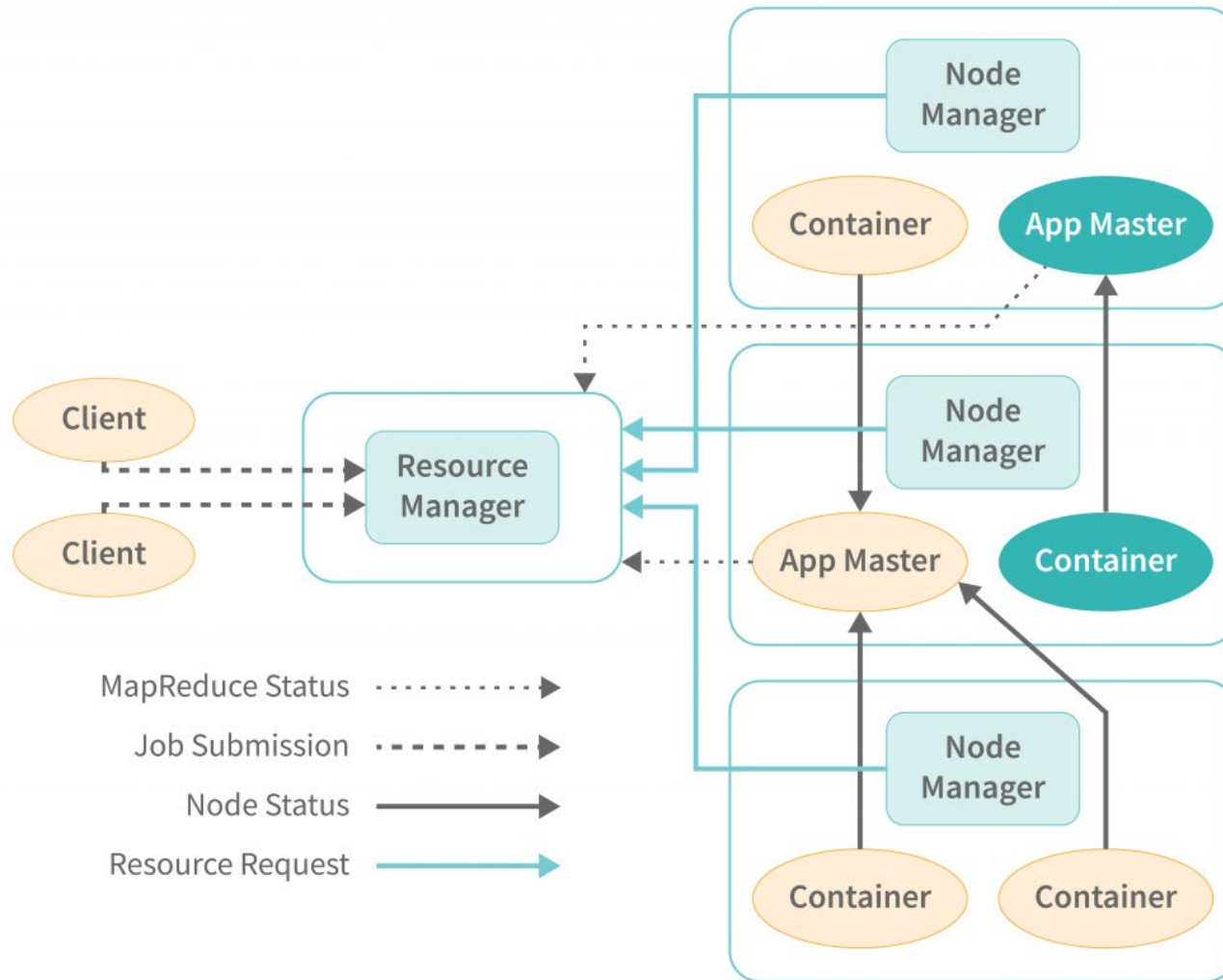
MapReduce v2 (MRv2): On YARN

- Architecture
 - **YARN (Yet Another Resource Negotiator)**
 - Separates resource management from job execution
 - **ResourceManager (RM): Global master**
 - Scheduler: manages cluster resources
 - ApplicationsManager: Application (job) life-cycle management
 - **NodeManager (NM): Per-node agent**
 - Manages containers (generalized resource units: CPU, memory)
 - **ApplicationMaster (AM): Per-job master**
 - Handles job-specific task scheduling and execution for MR
 - Job progress and status report
 - **Containers: Flexible resource unit**
 - Allocated by NodeManager (not fixed slots)

Hadoop YARN architecture



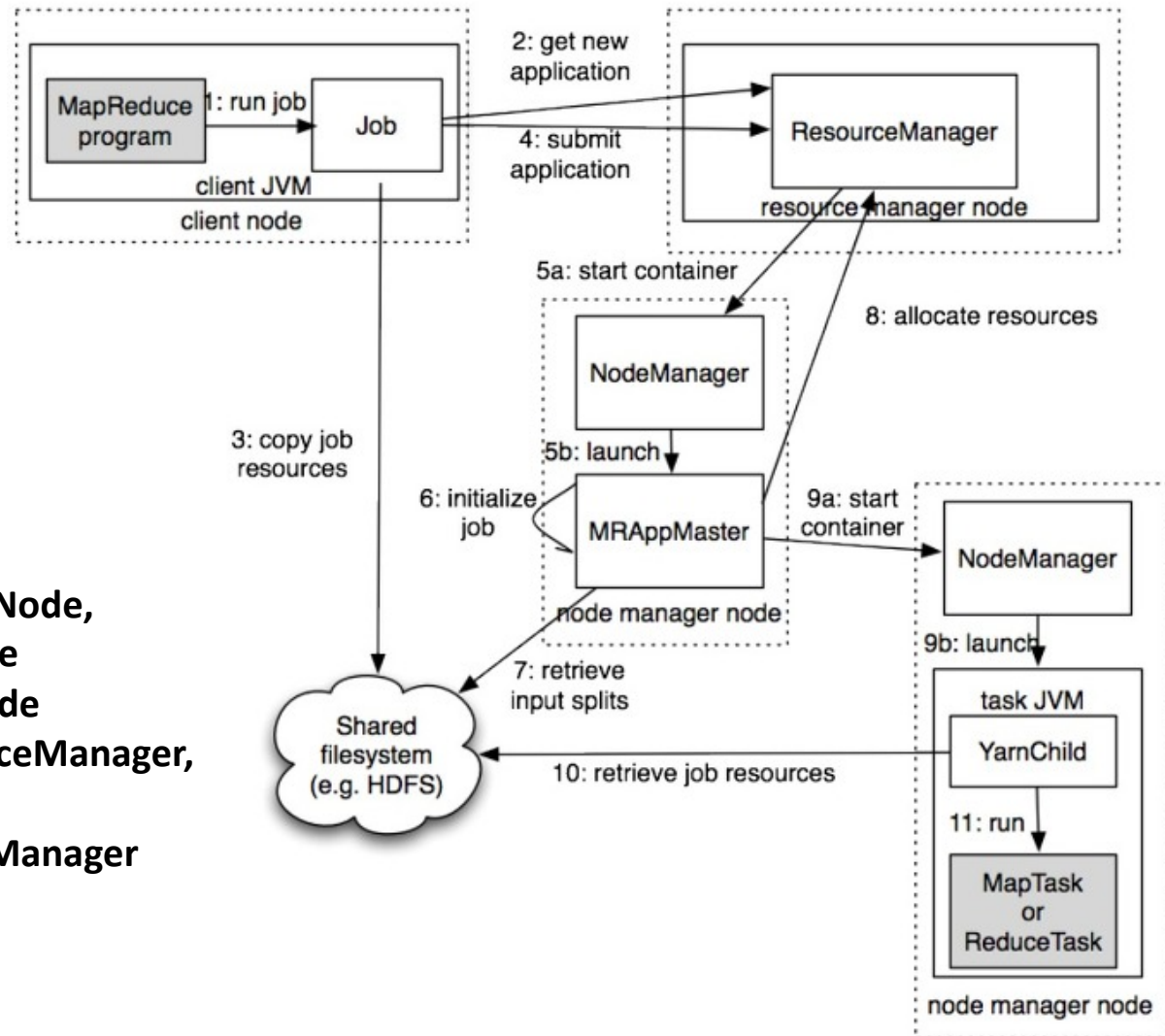
Hadoop YARN architecture



MapReduce v2 (MRv2)

- Workflow
 - Client submits job to ResourceManager.
 - RM starts an ApplicationMaster (AM) in a container.
 - ApplicationMaster
 - Requests containers from RM for map/reduce tasks.
 - Launches tasks on NodeManagers.
 - Each task runs inside a container (resources dynamically allocated).
 - Progress/status reported to AM (not RM).
 - AM handles task failures, rescheduling (if needed).
 - When job finishes, AM shuts down and resources release

Job Execution Workflow (MRv2)



V1 → V2

- NameNode → NameNode, Secondary NameNode
- DataNode → DataNode
- JobTracker → ResourceManager, ApplicationMaster
- TaskTracker → NodeManager

MapReduce v2 (MRv2)

- Advantages
 - Better scalability
 - Separation of resource management and job scheduling
 - High availability
 - No single JobTracker bottleneck
 - Flexible resource allocation
 - Not tied to fixed map/reduce slots
 - YARN supports multiple frameworks (MR, Spark, Storm, Flink, etc.)

Key differences (MR v1 vs. v2)

	MR v1	MR v2
Master daemon	JobTracker	ResourceManager + ApplicationMaster
Worker daemon	TaskTracker	NodeManager
Resource management	JobTracker	ResourceManager
Job Scheduling	JobTracker	ApplicationMaster
Resource unit	Fixed slots (map/reduce)	Flexible container (CPU + memory)
Scalability	Limited	Highly scalable
Framework support	Only MR	MR + Spark, Storm, Flink, etc.
Fault tolerance (reassign tasks)	JobTracker	ApplicationMaster



Let's build up a MapReduce job
example

The Scenario

- Build a WordCount
- Input data: Using the Shakespeare Poems
- Count the number of times a word appears in the data set
- Use Map and Reduce to do this work
- Step-by-step of creating the MR process

Driver Code: Setting up the MR Job

- A Job object forms the specification for the job
- Job needs to know:
 - The jar file that the code is in which will be distributed around the cluster: **setJarByClass()**
 - The input path(s) in HDFS for the job: **FileInputFormat.addInputPath()**
 - The output path(s) in HDFS for the job: **FileOutputFormat.setOutputPath()**
 - The Mapper and Reducer classes: **setMapperClass(), setReducerClass()**
 - The Reducer output key and value classes: **setOutputKeyClass(), setOutputValueClass()**
 - The Mapper output key and value classes if they are different from the Reducer: **setMapOutputKeyClass(), setMapOutputValueClass()**
 - The name of the job, default is the name of the jar file: **setJobName()**
- Default input considers the file as lines of text
 - The default key input is **LongWritable** (the byte offset into the file)

Driver code

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

public class WordCountDriver extends Configured implements Tool {

    @Override
    public int run(String[] args) throws Exception {
        Configuration conf = new Configuration();

        Job job = new Job(conf, "WordCount");
        job.setJarByClass(WordCountDriver.class);

        job.setMapperClass(WordCountMapper.class);
        job.setReducerClass(WordCountReducer.class);

        job.setInputFormatClass(TextInputFormat.class);

        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(LongWritable.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(LongWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        job.waitForCompletion(true);
        return 0;
    }

    public static void main(String[] args) throws Exception {
        ToolRunner.run(new WordCountDriver(), args);
    }
}
```

Driver code

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

public class WordCountDriver extends Configured implements Tool {

    @Override
    public int run(String[] args) throws Exception {
        Configuration conf = new Configuration();

        Job job = new Job(conf, "WordCount");
        job.setJarByClass(WordCountDriver.class);

        job.setMapperClass(WordCountMapper.class);
        job.setReducerClass(WordCountReducer.class);

        job.setInputFormatClass(TextInputFormat.class);

        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(LongWritable.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(LongWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        job.waitForCompletion(true);
        return 0;
    }

    public static void main(String[] args) throws Exception {
        ToolRunner.run(new WordCountDriver(), args);
    }
}
```

1. Create a new job, specify the class which will be called to run the job.
2. We can define job name optionally. E.g. `job.setJobName("WordCount");`



Driver code

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

public class WordCountDriver extends Configured implements Tool {

    @Override
    public int run(String[] args) throws Exception {
        Configuration conf = new Configuration();

        Job job = new Job(conf, "WordCount");
        job.setJarByClass(WordCountDriver.class);
        job.setMapperClass(WordCountMapper.class);
        job.setReducerClass(WordCountReducer.class);
        job.setInputFormatClass(TextInputFormat.class);

        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(LongWritable.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(LongWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        job.waitForCompletion(true);
        return 0;
    }

    public static void main(String[] args) throws Exception {
        ToolRunner.run(new WordCountDriver(), args);
    }
}
```

Give the job information about the classes for the Mapper and the Reducer.



Driver code

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

public class WordCountDriver extends Configured implements Tool {

    @Override
    public int run(String[] args) throws Exception {
        Configuration conf = new Configuration();

        Job job = new Job(conf, "WordCount");
        job.setJarByClass(WordCountDriver.class);

        job.setMapperClass(WordCountMapper.class);
        job.setReducerClass(WordCountReducer.class);
        job.setInputFormatClass(TextInputFormat.class);

        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(LongWritable.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(LongWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        job.waitForCompletion(true);
        return 0;
    }

    public static void main(String[] args) throws Exception {
        ToolRunner.run(new WordCountDriver(), args);
    }
}
```

This is default input format. E.g.,
InputFormat(TextInputFormat).



Driver code

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

public class WordCountDriver extends Configured implements Tool {

    @Override
    public int run(String[] args) throws Exception {
        Configuration conf = new Configuration();

        Job job = new Job(conf, "WordCount");
        job.setJarByClass(WordCountDriver.class);

        job.setMapperClass(WordCountMapper.class);
        job.setReducerClass(WordCountReducer.class);

        job.setInputFormatClass(TextInputFormat.class);

        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(LongWritable.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(LongWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        job.waitForCompletion(true);
        return 0;
    }

    public static void main(String[] args) throws Exception {
        ToolRunner.run(new WordCountDriver(), args);
    }
}
```

Specify the format of the intermediate output key and value produced by the Mapper



Driver code

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

public class WordCountDriver extends Configured implements Tool {

    @Override
    public int run(String[] args) throws Exception {
        Configuration conf = new Configuration();

        Job job = new Job(conf, "WordCount");
        job.setJarByClass(WordCountDriver.class);

        job.setMapperClass(WordCountMapper.class);
        job.setReducerClass(WordCountReducer.class);

        job.setInputFormatClass(TextInputFormat.class);

        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(LongWritable.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(LongWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        job.waitForCompletion(true);
        return 0;
    }

    public static void main(String[] args) throws Exception {
        ToolRunner.run(new WordCountDriver(), args);
    }
}
```

Specify the types for the Reducer output key and value.



Driver code

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

public class WordCountDriver extends Configured implements Tool {

    @Override
    public int run(String[] args) throws Exception {
        Configuration conf = new Configuration();

        Job job = new Job(conf, "WordCount");
        job.setJarByClass(WordCountDriver.class);

        job.setMapperClass(WordCountMapper.class);
        job.setReducerClass(WordCountReducer.class);

        job.setInputFormatClass(TextInputFormat.class);

        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(LongWritable.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(LongWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        job.waitForCompletion(true);
        return 0;
    }

    public static void main(String[] args) throws Exception {
        ToolRunner.run(new WordCountDriver(), args);
    }
}
```

Specify the input directory (where the data will be read from) and output directory (where the data will be written).



Driver code

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

public class WordCountDriver extends Configured implements Tool {

    @Override
    public int run(String[] args) throws Exception {
        Configuration conf = new Configuration();

        Job job = new Job(conf, "WordCount");
        job.setJarByClass(WordCountDriver.class);

        job.setMapperClass(WordCountMapper.class);
        job.setReducerClass(WordCountReducer.class);

        job.setInputFormatClass(TextInputFormat.class);

        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(LongWritable.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(LongWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        job.waitForCompletion(true);
        return 0;
    }

    public static void main(String[] args) throws Exception {
        ToolRunner.run(new WordCountDriver(), args);
    }
}
```

Submit the job and wait for completion.

Mapper

- The mapper may use or ignore the input key
 - E.g., a standard pattern is to read a file one line at a time.
 - Key = byte offset into the file where the line starts
 - Value= contents of the line in the file
 - Typically the key can be considered irrelevant
- Mapper class extends the Mapper $\langle K1, V1, K2, V2 \rangle$ class
 - Key and value classes implement the WritableComparable and Writable interfaces.
 - Most Mappers override the map() method which is called once for every KV pair in the input.
 - In general, the map input types K1, V1 are different from the map output types K2, V2.

Mapper Code

```
import java.io.IOException;
import java.util.StringTokenizer;
```

```
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
```

```
public class WordCountMapper extends
```

```
    Mapper<LongWritable, Text, Text, LongWritable> {
    private final static LongWritable one = new LongWritable(1);
    private Text word = new Text();
```

```
    @Override
```

```
    public void map(LongWritable inputKey, Text inputVal, Context context)
        throws IOException, InterruptedException {
```

```
        String line = inputVal.toString();
        StringTokenizer tokenizer = new StringTokenizer(line);
        while (tokenizer.hasMoreTokens()) {
            word.set(tokenizer.nextToken());
            context.write(word, one);
```

```
        }
```

```
    }
```

```
}
```

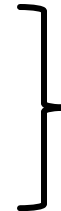
Inputs



Outputs



Processes the input
text



Write the outputs



What the mapper does

- Input to the Mapper:

```
("this one I think is called a yink")  
("he likes to wink, he likes to drink")  
("he likes to drink and drink and drink")
```

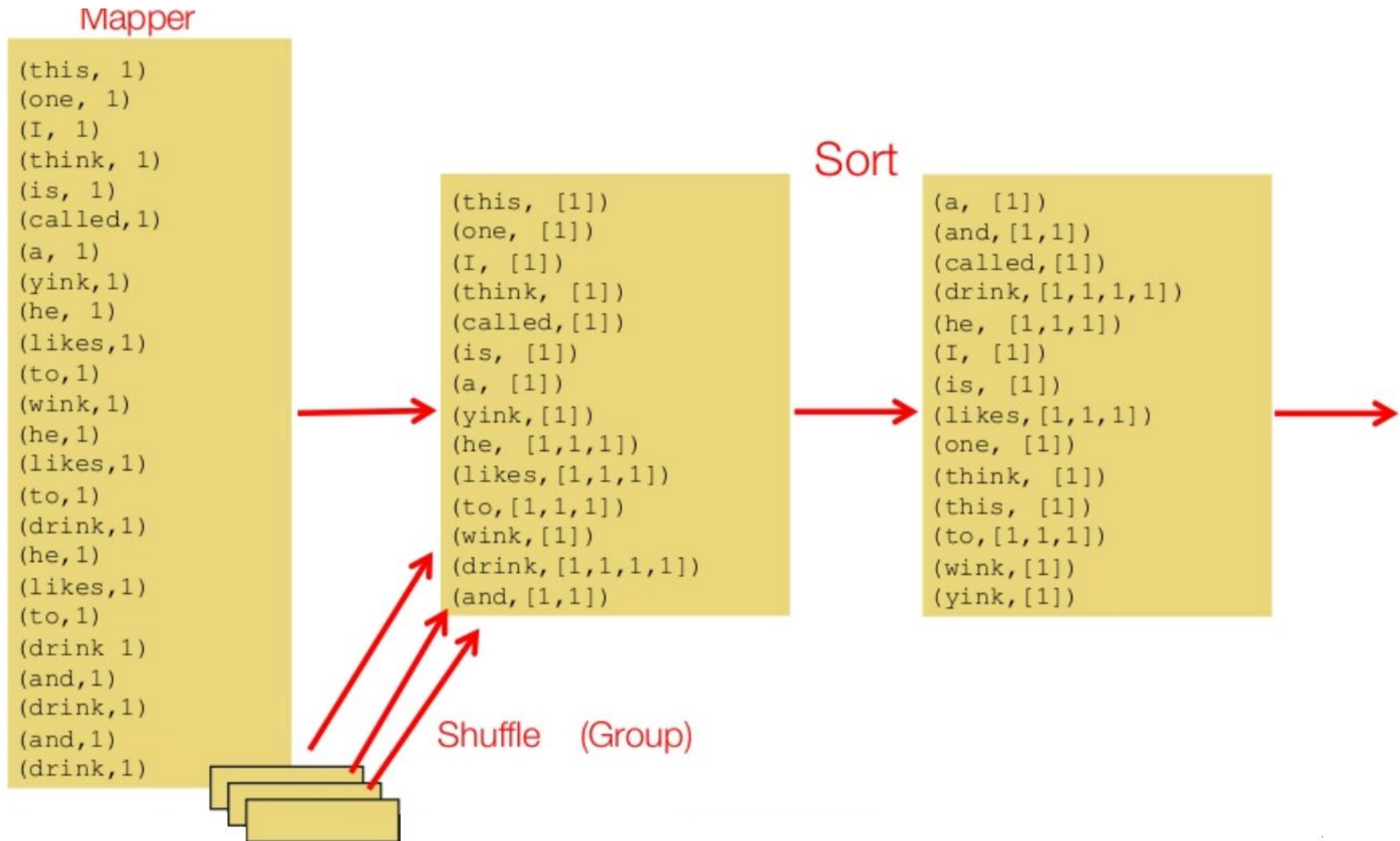
- Output from the Mapper:

```
(this, 1)  
(one, 1)  
(I, 1)  
(think, 1)  
(is, 1)  
(called, 1)  
(a, 1)  
(yink, 1)  
(he, 1)  
(likes, 1)  
(to, 1)  
(wink, 1)  
(he, 1)  
(likes, 1)  
(to, 1)  
(drink, 1)  
(he, 1)  
(likes, 1)  
(to, 1)  
(drink 1)  
(and, 1)  
(drink, 1)  
(and, 1)  
(drink, 1)
```

Shuffle and Sort

- Shuffle
 - Integrates the data (KV pairs) from each mapper output
 - For now, integrates into 1 file
- Sort
 - The set of intermediate keys on a single node is automatically sorted by Hadoop before they are presented to the Reducer
 - Sorted by key
- **Partitioner** in Mapper determines what subset of data goes to which Reducer

What Shuffle and Sort do



Reducer

- Reducer class extends Reducer <K2, V2, K3, V3> class
 - Key and value classes implement the WritableComparable and Writable interfaces
 - Called once for each input key
 - Generates a list of output KV pairs by iterating over the values associated with the input key
 - Reduce input types K2, V2 must be the same types as the map output types
 - Reduce output types K3, V3 can be different from the reduce input types

Reducer Code

```
import java.io.IOException;

import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class WordCountReducer extends
    Reducer<Text, LongWritable, Text, LongWritable> {

    @Override
    public void reduce(Text key, Iterable<LongWritable> listOfValues,
        Context context) throws IOException, InterruptedException {
        long sum = 0;
        for (LongWritable val : listOfValues) {
            sum = sum + val.get();
        }
        context.write(key, new LongWritable(sum));
    }
}
```

Inputs →

Outputs ←

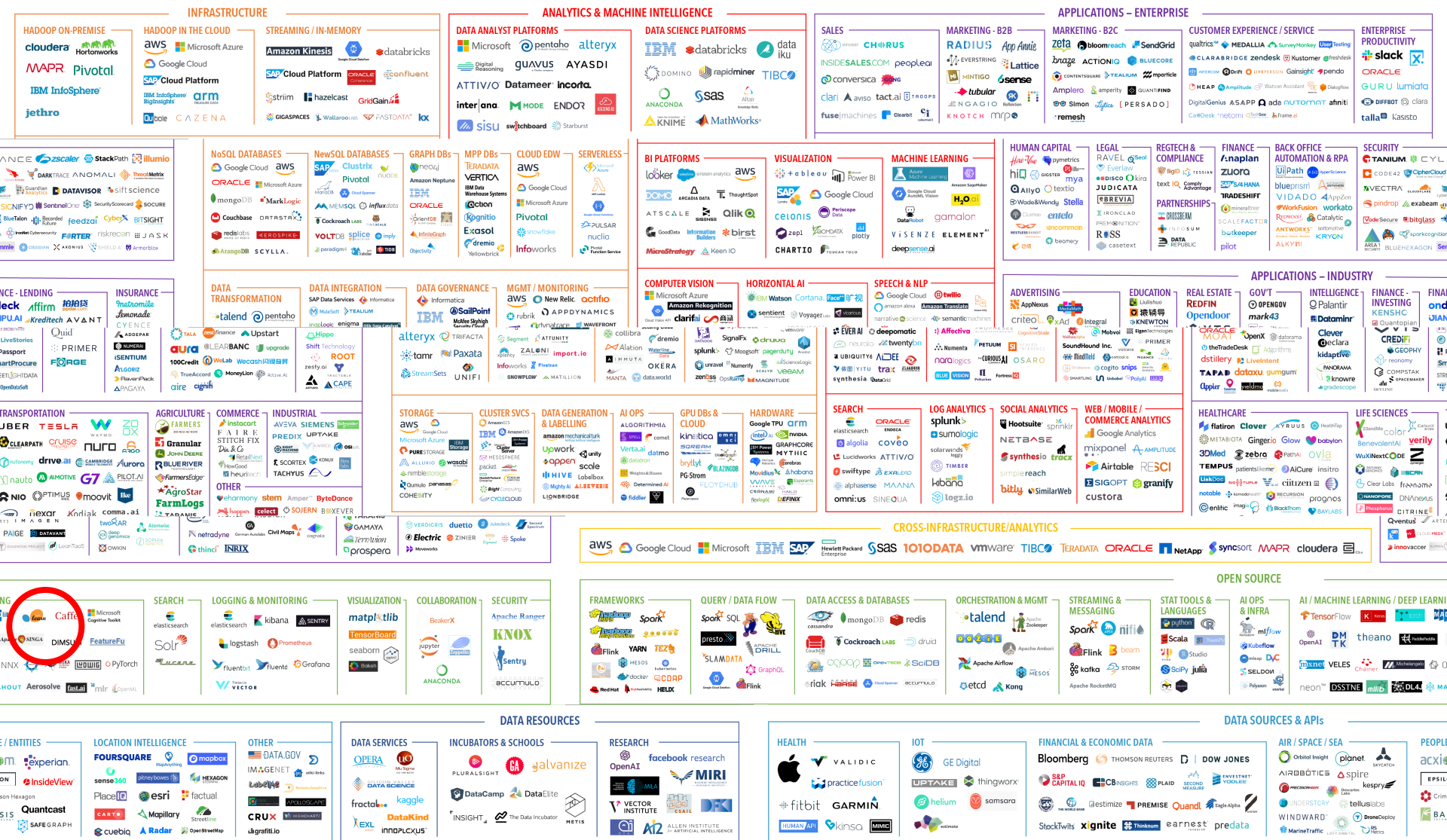
Processes the input text

Write the outputs →

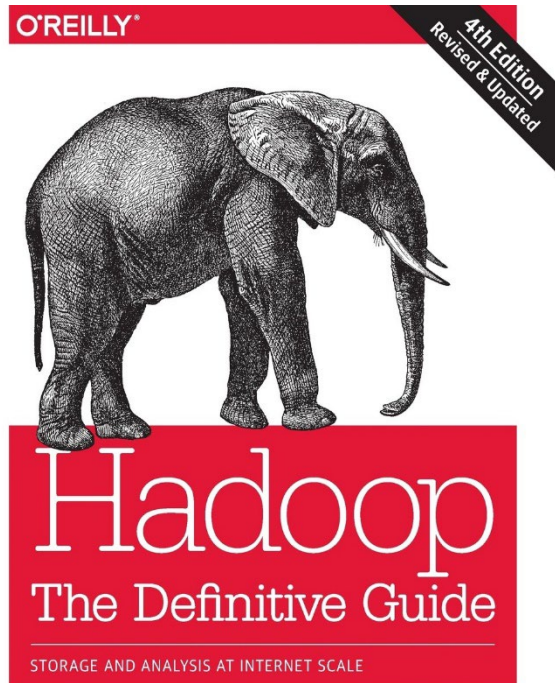


Hadoop summary

- Distributed frame work for processing and storing data generally on commodity hardware. Completely open source and written in Java.
- **Store anything**
 - Unstructured or semi structured data,
- **Storage capacity**
 - Scale linearly, cost in not exponential.
- **Data locality and process in your way.**
 - Code moves to data
 - In MR you specify the actual steps in processing the data and drive the output.
- **Failure and fault tolerance:**
 - Detect Failure and Heals itself.
 - Reliable, data replicated, failed task are rerun , no need maintain backup of data
- **Cost effective**
 - A scale-out architecture operating on a cluster of commodity PC machines.
- Primarily used for batch processing, not real-time/ transactional user applications.



Questions?



Tom White

■ References

- Hadoop: The Definitive Guide, Third(or fourth) Edition by Tom White
- <http://Hadoop.apache.org>